INTELLIGENT LIBRARY SEARCH AND BOOK COLLECTION AGENT (G-172)

Submitted by

Diya Dasauni (1000023103)

Ekant Chandna (1000024196)

Krishna Dhananjay Gupta (1000024203)

Md. Abbas Raza (1000024226)

Under the Supervision of

Dr. Neha Sharma
(Assistant Professor)



DIT UNIVERSITY, DEHRADUN

(State Private University through State Legislature Act No. 10 of 2013 of Uttarakhand and approved by UGC)
Mussoorie Diversion Road, Dehradun, Uttarakhand – 248009, India.

December, 2026

CANDIDATE DECLARATION

We declare that this written submission represents our ideas in our own words and where other ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed. The plagiarism check report is attached at the end of this document.

The matter presented in this report has not been submitted by us for the award of any other degree or course of this or any other institute.

Diya Dasauni (1000023103) Signature and Date _____

Ekant Chandna (1000024196) Signature and Date _____

Krishna Dhananjay Gupta (1000024203) Signature and Date _____

Md. Abbas Raza (1000024226) Signature and Date _____

Dr. Neha Sharma (Assistant Professor) Signature and Date _____
School of Computing, DIT University, Dehradun

Table of Contents

Candidate Declaration **1**
1 Introduction **3**
1.1 Artificial Intelligence, State-Space Search and Intelligent Agents 3
1.2 Why Graph Search Suits Library Navigation 3
1.3 Scope of the Project 3
1.4 Organisation of the Report 4
2 Project Description **5**
2.1 Purpose 5
2.2 Problem Statement 5
2.3 System Requirements 5
2.3.1 Hardware Requirements 5
2.3.2 Software Requirements 6
2.3.3 Frameworks and Technologies 6
2.3.4 Library Knowledge Base and Data Representation 6
2.3.5 Application Programming Interface 7
2.3.6 Installation and Execution 7
3 Methods and Implementations **9**
3.1 System Architecture 9
3.2 Methodology: From Input to Output 9
3.2.1 Graph Representation 10
3.2.2 Library Nodes, Edges, and Book Data 10
3.2.3 Rules and Constraints 11
3.3 Search Algorithms 12
3.3.1 Breadth-First Search (BFS) 12
3.3.2 Depth-First Search (DFS) 13
3.3.3 A* Search 13
3.3.4 Heuristic Function and Path-Cost Calculation 14
3.3.5 Comparison of Search Algorithms 14
3.4 Intelligent Agent and Multi-Book Planning 15
3.4.1 Goal-Based Search Behaviour 15
3.4.2 Priority-Based Book Collection 15
3.4.3 Greedy Nearest-Neighbor Route Planning 15
3.4.4 Time/Cost Limit Handling 16
3.5 Library Navigation and Collection Management 16
3.6 Implementation 17
3.6.1 Backend Implementation 17
3.6.2 Frontend Implementation 18
3.6.3 API Communication 18
3.6.4 Graph Visualization 18
3.6.5 Testing and Error Handling 18
3.7 Source Code 18
3.7.1 Important Backend Code 19
3.7.2 Search Algorithm Implementations 20
3.7.3 Multi-Book Planning Implementation 23
3.7.4 Frontend and API Integration 24
3.7.5 Unit Tests 27
3.8 Output Screenshots 27
4 Result Analysis **35**
4.1 Test Environment 35
4.2 Single-Book Navigation Scenarios 35
4.3 BFS, DFS, and A* Behaviour 36
4.4 Multi-Book Collection Scenarios 37
4.5 Priority and Time/Cost Limit Analysis 38
4.6 Restricted Access and Closed-Section Scenarios 39
4.7 Unit Test Results 39
4.8 Observations 40
4.9 Limitations Observed 40
5 Conclusion **42**
Future Enhancements **43**
Bibliography **44**

1 Introduction

Finding a particular book in a large physical library is a navigation problem as much as a search problem. A reader may know the title and the subject area but not the shelf, and the route from the entrance to that shelf may depend on which sections are open, which shelves require special permission and how far apart the sections are. When a reader wants several books, the order in which they are visited also changes the total walking effort. The **Intelligent Library Search and Book Collection Agent** is a small academic project that turns this situation into a computational problem and solves it with classical Artificial Intelligence (AI) search techniques.

The project models the library as a weighted graph. Locations such as the entrance and the subject sections are nodes, and walkways between them are edges with a distance and a time. A web interface lets the user search for a book, add available books to a personal collection, and ask the system for a route. The route can be found with Breadth-First Search (BFS), Depth-First Search (DFS) or A* search, and a simple planning agent can combine several books into one trip while honouring book priorities and a time or cost limit.

1.1 Artificial Intelligence, State-Space Search and Intelligent Agents

In the textbook view of AI, many problems can be described as a search through a **state space** [1]. A problem is defined by an initial state, the actions available in each state, a transition model that says which state results from an action, a goal test, and a path cost. A solution is a sequence of actions that leads from the initial state to a goal state. The library problem fits this definition directly: a state is the reader’s current location, an action is walking along an edge to a connected location, the goal test checks whether the current location is the shelf of the wanted book, and the path cost is the total distance or time walked.

Uninformed search strategies such as BFS and DFS use only the problem definition. **Informed** or **heuristic** strategies such as A* additionally use a heuristic function h(n) that estimates the remaining cost from a node n to the goal [2], [1]. In this project the heuristic is the straight-line (Euclidean) distance between the coordinates of two nodes. Because A* combines the cost already paid with this estimate of the cost still to pay, it normally explores fewer nodes than an uninformed method while still finding a least-cost route when the heuristic is admissible.

An **intelligent agent** perceives its environment and chooses actions to achieve its goals [1]. The multi-book planner in this project is a simple goal-based agent: its percepts are the collection list, the library graph and the time limit, its goals are the shelves of the requested books, and its actions are the walking routes returned by A*. It is deliberately modest. It does not learn, it does not use machine learning or any language model, and it does not call any external AI service; all of its behaviour comes from explicit search code and a small rule for ordering books.

1.2 Why Graph Search Suits Library Navigation

A library floor plan is naturally a graph: sections are vertices, aisles are edges, and the cost of an aisle is its length or its walking time. Graph algorithms are well understood, easy to test on small data and easy to visualise, which makes them a good educational vehicle. Using a weighted graph also makes an important distinction visible: a route with the fewest edges (what BFS finds) is not necessarily the route with the least walking distance (what A* with an admissible heuristic finds) [3], [1]. The project lets a student observe this difference on one data set.

1.3 Scope of the Project

The project is an academic demonstration. It consists of a Flask back end written in Python, a JSON file holding the library graph and book list, and a front end built with plain HTML, CSS and JavaScript, with Vis.js used to draw the graph [4], [5]. Within that scope the report covers the graph model, the three search algorithms, the heuristic, the priority-based multi-book planner, the HTTP interface and the unit tests.

The project does **not** attempt to provide a real library catalogue integration, a database, user accounts or authentication, indoor positioning, live occupancy data, or a globally optimal multi-stop route. The library map is a small hand-written example with nine locations. Some features described in the project's README are only partly present in the code; this report states clearly which features are implemented, which are only partly implemented and which are absent (see Section 3.2.3 and Chapter 4).

1.4 Organisation of the Report

Chapter 2 describes the purpose, problem statement, requirements and execution steps. Chapter 3 explains the architecture, the graph model, the algorithms, the planning agent and the implementation, with source-code excerpts and application screenshots. Chapter 4 analyses results obtained by actually running the code and its tests, and records the limitations observed. Chapter 5 concludes, followed by proposed future enhancements and the bibliography.

2 Project Description

2.1 Purpose

The purpose of the project is to demonstrate how classical AI search can support library navigation and book collection. Specifically, the system is intended to:

- represent a library as a weighted graph of locations and walkways;
- search for a book by title or author and report its section and availability;
- maintain a collection list of available books without duplicates;
- find a route from the entrance to a book's shelf using BFS, DFS or A*;
- use a Euclidean-distance heuristic in A* and report path cost and the number of explored nodes;
- plan one trip for several books in priority order within a user-defined time or cost limit; and
- visualise the library graph and highlight the calculated route in the browser.

These functions follow the normal workflow of a reader: search, select, collect and walk. The report also records, honestly, where the delivered code falls short of this intent.

2.2 Problem Statement

Locating books in a physical library raises several difficulties:

- a reader may not know where a book is shelved or how to reach that shelf;
- the shortest route by number of corridors is not always the shortest in walking distance;
- some sections may be closed, and some shelves may be restricted to authorised readers;
- a reader collecting several books must decide which to fetch first, and may have only limited time; and
- unavailable books should not be offered as targets for a trip.

The project addresses these difficulties with a graph model and search algorithms. Given a start state (the entrance) and a goal state (the shelf of a selected book), the system finds a path and reports its cost. Given several books, a priority-ordered greedy planner builds one combined route and lists any books it could not fit within the limit.

2.3 System Requirements

The project is small and has no demanding resource needs. The figures below are what the project files state or imply; where this report offers a practical suggestion that the project itself does not establish, it says so. No performance benchmark was established for this report.

2.3.1 Hardware Requirements

Table 1: Suggested hardware for running the project (practical suggestions, not project-mandated minimums)

Component	Suggestion
Processor	Any modern computer able to run Python 3 and a current web browser. The project does not state a minimum.
Memory	No requirement is stated. The data set is a 360-line JSON file, so memory use by the application is small.
Display	The interface uses a fixed 350 px sidebar beside a graph panel, so a desktop or laptop screen is recommended over a phone screen.

Component	Suggestion
Network	Internet access is needed on the machine that opens the page, because `templates/index.html` loads Vis.js from the unpkg CDN (`https://unpkg.com/vis-network/standalone/umd/vis-network.min.js`). Without it the graph cannot be drawn.

2.3.2 Software Requirements

Table 2: Software requirements

Item	Details
Operating system	Not restricted; any system that runs Python 3 and a browser.
Python	Python 3 (README). The bundled bytecode caches were produced by CPython 3.14; this report's tests were run on Python 3.12.3.
Web framework	Flask, pinned as `Flask==3.0.0` in `requirements.txt`. This report's runs used Flask 3.1.3; version 3.0.0 itself was not tested.
Python standard library	`heapq`, `math`, `json`, `os` and `unittest`.
Front end	HTML, CSS and plain JavaScript; no framework, build step, database or login (README, confirmed by inspection of the files).
Graph visualisation	Vis.js `vis-network`, loaded from the unpkg CDN without a pinned version. A local copy of version 10.1.4 was used for this report's screenshots (see Section 4.1).
Browser	Any modern browser. Screenshots in this report were taken with headless Chromium.

2.3.3 Frameworks and Technologies

Table 3 lists each technology and its purpose in the project.

Table 3: Technologies and their purpose

Technology	Purpose in this project
Python	Implements the graph rebuild, BFS, DFS, A*, the heuristic and the multi-book planner.
Flask [4]	Serves the page and provides the JSON endpoints `/api/data`, `/api/navigate` and `/api/multi_route`.
`heapq` [6]	Binary-heap priority queue used as the A* open list.
`math` [7]	`math.sqrt` computes the Euclidean heuristic.
JSON [8]	Format of `library_data.json` and of all request and response bodies.
JavaScript `fetch` [9]	Sends requests from the browser to the Flask endpoints.
Vis.js [5]	Draws section nodes, book nodes and edges, and highlights the route.
`unittest` [10]	Runs the algorithm unit tests in `test_algorithm.py`.

2.3.4 Library Knowledge Base and Data Representation

All library knowledge is stored in `JSON/library_data.json`, which has three top-level keys. Table 4 summarises them, and Listing 1 shows the shape of one record of each kind.

Table 4: Structure of library_data.json (counts verified from the file)

Key	Count	Contents
`nodes`	9	An object keyed by location name. Each value has `x`, `y` (coordinates used by the heuristic and the drawing) and `restricted` (true or false).

Key	Count	Contents
`edges`	12	A list of objects with `from`, `to`, `distance` and `time`. Edges are treated as undirected when the graph is rebuilt.
`books`	30	A list of objects with `id`, `title`, `author`, `category`, `shelf`, `available` and `priority`. `shelf` names the node where the book is located.

Listing 1: Shape of one node, one edge and one book record in library_data.json

```
"CS_Section": { "x": 2, "y": 2, "restricted": false }
{ "from": "Entrance", "to": "CS_Section", "distance": 10, "time": 10 }
{ "id": "101", "title": "Introduction to Algorithms", "author": "Cormen",
  "category": "CS", "shelf": "CS_Section", "available": true, "priority": "High" }
```

In the supplied data all 30 books have `available` set to true, 14 have priority High and 16 have priority Low; no book has priority Medium although the planner supports it.

2.3.5 Application Programming Interface

The Flask application exposes one page route and three JSON routes (Table 5). All were exercised during this project's testing; the error behaviour listed is what was actually observed.

Table 5: Flask routes of the application

Route	Method	Request body	Response and observed behaviour
`/`	GET	—	Returns `index.html`.
`/api/data`	GET	—	Returns the whole `library_data.json` content (about 5.6 kB).
`/api/navigate`	POST	`target`, `algorithm` ("A*", "BFS" or "DFS"), `optimize` ("distance" or "time")	A*: `path`, `cost`, `explored`, `closed_list`. BFS and DFS requests returned HTTP 500 in every test (Section 4.3). An unreachable A* target returns `cost: Infinity`. An unknown target returns HTTP 500.
`/api/multi_route`	POST	`books` (full book objects), `time_limit`, `optimize`, optional `authorized`	`path`, `distance`, `explored`, `collected` (titles), `uncollected` (titles). A null time limit returned HTTP 500.

2.3.6 Installation and Execution

The README gives the following procedure, which is reproduced here with the caveat that this report's own runs started the server slightly differently (Section 4.1).

Table 6: Installation and execution steps

Step	Action
1	Install Python 3.
2	In the `AI-project` folder run `pip install -r requirements.txt` (installs Flask).
3	Change to the `python-files` folder with `cd python-files`.
4	Start the server with `python app.py` (the script calls `app.run(debug=True)`, so Flask's default address and port apply).
5	Open `http://127.0.0.1:5000` in a browser with internet access for the Vis.js CDN.
6	Run the tests with `python -m unittest test_algorithm.py -v` from `python-files`.
7	Stop the server with Ctrl+C.

3 Methods and Implementations

3.1 System Architecture

The application follows a conventional client–server design (Figure 1). The browser holds the user interface and the user's collection list in memory. The Flask application loads `library_data.json` once at start-up, rebuilds an adjacency dictionary, and answers three kinds of request: serving the page, returning the raw data, and computing routes. Route computation is delegated to `algorithms.py` (BFS, DFS, A* and the heuristic) and to `multi_book_agent.py` (the planner, which itself calls A*). Results travel back as JSON and are rendered by `script.js`, which uses Vis.js to draw the graph and highlight the route.

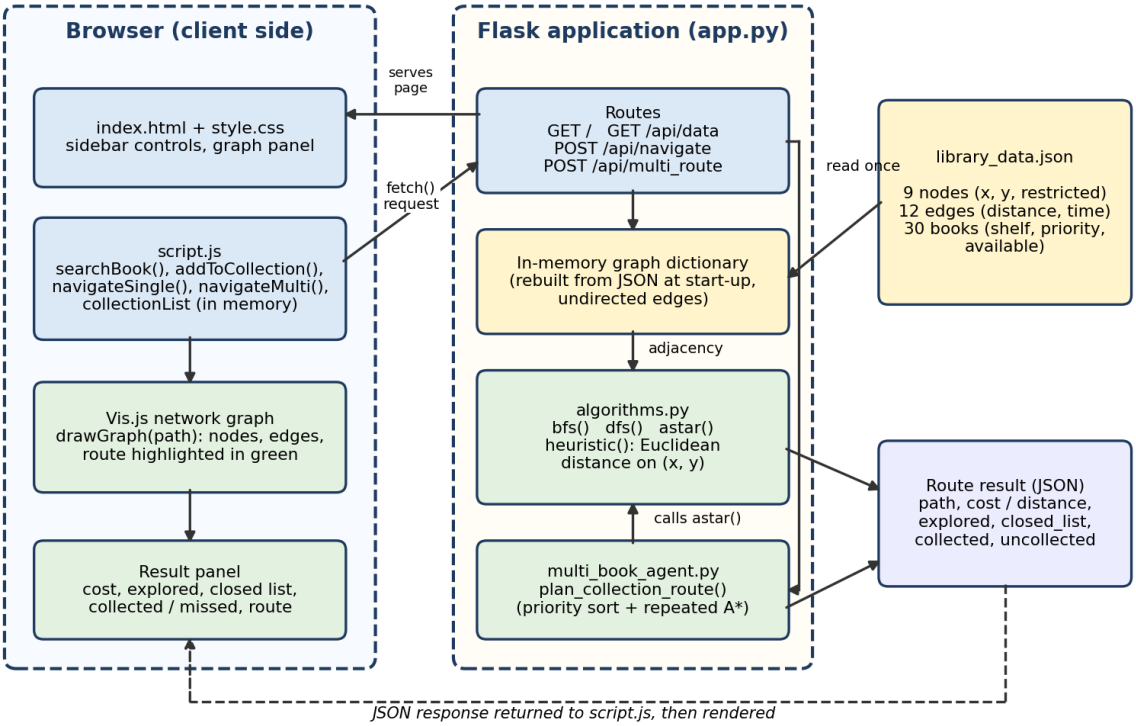


Figure 1: Architecture of the Intelligent Library Search and Book Collection Agent

Two architectural facts matter for the rest of this report. First, **searching for a book happens entirely in the browser**: `script.js` scans the book list it received from `/api/data`, and no server endpoint performs a search. Second, **there is no persistent state on the server**: the collection list lives in a JavaScript array and is lost when the page is reloaded, and every route request carries all the information the server needs.

3.2 Methodology: From Input to Output

A typical session proceeds as follows. The page loads and fetches `/api/data`; Vis.js draws the graph. The user types a title or author and presses "Find Book"; the first matching book is shown with its shelf and availability. The user adds the book to the collection. For a single-book route the user selects an algorithm and presses "Navigate Single Book"; for a trip the user sets an optimisation mode and a limit and presses "Plan Multi-Book Route". The server returns a path and cost, and the browser draws the path in green and prints the figures in the Results panel.

3.2.1 Graph Representation

In the Python back end the graph is a dictionary of dictionaries: `graph[u][v]` holds `{"distance": d, "time": t}` for every edge between u and v. Each edge in the JSON file is inserted in both directions, so the graph is undirected. A second dictionary, `nodes_data`, holds the coordinates and the `restricted` flag of each node and is used by the heuristic and by the restriction rule. Figure 2 shows the nine nodes at their stored coordinates with edge distances.

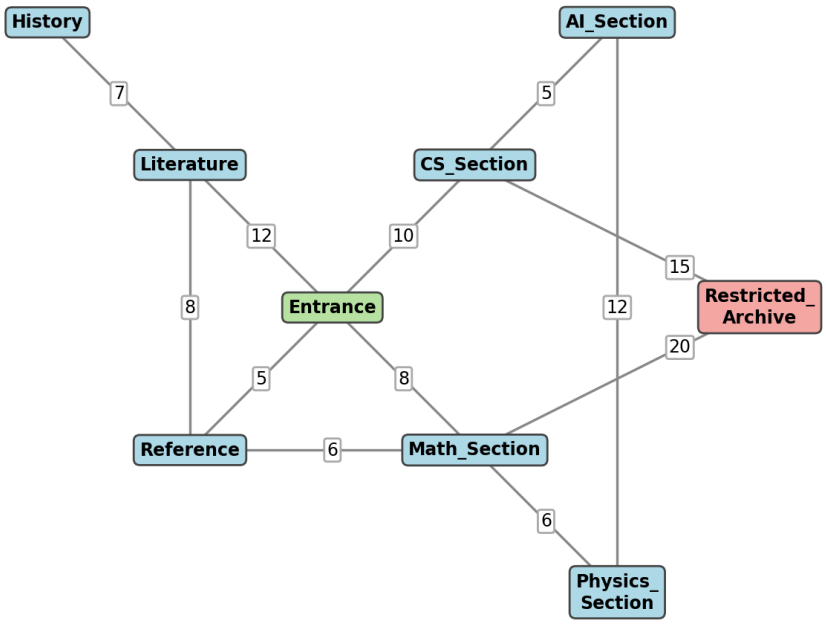


Figure 2: The library graph drawn from library_data.json (node positions are the stored coordinates; edge labels are distances; the red node is restricted)

3.2.2 Library Nodes, Edges, and Book Data

Table 7 and Table 8 reproduce the graph data. The Entrance is the start state of every route. Only `Restricted_Archive` has `restricted` set to true.

Table 7: Nodes in library_data.json

Node	x	y	Restricted	Books shelved (High / Low)
Entrance	0	0	No	0
CS_Section	2	2	No	4 (2 / 2)
AI_Section	4	4	No	4 (3 / 1)
Math_Section	2	-2	No	4 (2 / 2)
Physics_Section	4	-4	No	4 (2 / 2)
Literature	-2	2	No	4 (1 / 3)
History	-4	4	No	4 (2 / 2)
Reference	-2	-2	No	3 (1 / 2)
Restricted_Archive	6	0	Yes	3 (1 / 2)

Table 8: Edges in library_data.json (distance equals time on every edge)

From	To	Distance	Time
Entrance	CS_Section	10	10
Entrance	Math_Section	8	8
Entrance	Literature	12	12
Entrance	Reference	5	5
CS_Section	AI_Section	5	5
CS_Section	Restricted_Archive	15	15
Math_Section	Physics_Section	6	6
Math_Section	Restricted_Archive	20	20
Math_Section	Reference	6	6
AI_Section	Physics_Section	12	12
Literature	History	7	7
Literature	Reference	8	8

Each book record names its shelf, and that shelf is a section node. **The books themselves are not nodes of the Python graph.** The README states that each book is also a node connected to its section; in the delivered code the book nodes exist only in the browser, where `drawGraph()` adds them as small ellipses linked to their section by dashed lines purely for display. A route therefore ends at the book's section, not at an individual book position. Table 9 summarises the book data.

Table 9: Book data summary

Property	Value	Notes
Number of books	30	Per-section counts are given in the nodes table above.
`available` = true	30	No unavailable book exists in the supplied data.
Priority High / Low	14 / 16	No Medium book; the planner nevertheless maps Medium to rank 2.
Restricted shelf	3 books	Books 104, 117 and 118 are shelved at `Restricted_Archive`.

3.2.3 Rules and Constraints

The project describes several rules: availability, restricted shelves, closed sections, crowded sections, priorities and a time limit. Because the README describes more than the code does, each rule was checked against the source and against actual runs. Table 10 classifies every feature as **Implemented** (present and verified), **Partial** (present but incomplete), **Defective** (present but failing when exercised), **Not implemented**, or **Documented, unverified**.

Table 10: Implementation status of documented features (verified by inspection and execution)

Feature	Status	Evidence
A* search with f(n)=g(n)+h(n)	Implemented	`astar()` in `algorithms.py` (lines 56–90); results verified for all targets.
BFS and DFS functions	Implemented	Return correct paths when called directly (Section 4.3).
BFS and DFS from the web interface	Defective	`app.py` unpacks 3 values from 2-value functions (lines 41 and 44); every request returned HTTP 500.
Euclidean heuristic	Implemented	`heuristic()` lines 4–8; used only in distance mode. In time mode h = 0.

Feature	Status	Evidence
Search by title or author	Implemented	Client-side, first match only (`script.js` lines 14–44).
Availability check	Partial	Implemented in the browser, but all 30 books are available so the unavailable path was never exercised with real data.
Duplicate prevention	Implemented	Verified in the browser (Figure 15).
Priority-ordered planning	Implemented	`plan_collection_route()` sorts by priority; verified (Section 4.4).
Time or cost limit	Implemented	Verified; a limit sweep is given in Section 4.5.
Restricted shelves	Partial	Enforced only in A*. BFS and DFS ignore the flag. The interface never sends `authorized`, so restricted books can never be collected from the UI.
Closed sections	Partial	The three search functions accept a `closed_sections` list, but the API passes an empty list and no data or control sets it. Verified only by direct function calls.
Crowded sections	Not implemented	No crowd data or penalty exists in the code. The dropdown label “Fastest Time (Crowded Aisles)” only selects the `time` weights.
Books as graph nodes	Partial	Visual nodes only; the back-end graph has 9 nodes.
Display of open and closed lists	Partial	Only a closed list (visited set, unordered) is returned and only A* returns it successfully.
Unreachable-target handling	Defective	A* returns infinite cost; the API emits `Infinity`, which is not valid JSON [8], so the browser fails to parse it.
Run with `python app.py` on Flask 3.0.0	Documented, unverified	The report’s runs used Flask 3.1.3 (Section 4.1).
Unit tests	Partial	5 tests; 1 passes, 4 error (Section 4.7).

3.3 Search Algorithms

All three algorithms solve the same problem: given the graph, a start node (always the Entrance in the web application) and a goal node (the shelf section), return a path. They differ in the order in which they examine nodes and in whether they use edge costs. In every algorithm the number of nodes that were expanded before the goal was reached is returned as `explored`. The goal node itself is not counted.

3.3.1 Breadth-First Search (BFS)

Purpose. BFS finds a path with the fewest edges. **Data structures.** A Python list used as a first-in, first-out queue of complete paths, and a `visited` set. **Steps.** BFS removes the oldest path from the queue; if its last node is the goal it returns that path; otherwise, if the node has not been visited, it marks the node visited, increments the explored counter and appends one extended path for every neighbour that is not in the closed-section list. **Output.** The path and the explored count (two values) [1], [3]. The implementation is shown in Listing 6.

Strengths and limitations. BFS is simple and, on an unweighted graph, returns a shortest path in terms of edges. Here, however, edges carry different distances, and BFS ignores them, so the path it returns need not be the shortest in walking distance. Using a Python list with `pop(0)` makes each dequeue linear in the queue length; this is unimportant on nine nodes. The `authorized` parameter is accepted but never used, so BFS does not respect restricted nodes.

3.3.2 Depth-First Search (DFS)

Purpose. DFS follows one branch as deeply as possible before backtracking. **Data structures.** A list used as a last-in, first-out stack of paths and a `visited` set. **Steps.** The procedure mirrors BFS, but the *newest* path is removed first (`stack.pop()`). Because neighbours are pushed in the order they appear in the adjacency dictionary, the last neighbour inserted is examined first. **Output.** The path and the explored count (Listing 7).

Strengths and limitations. DFS needs little memory and can find a path quickly, but the path depends on neighbour order and is not guaranteed to be short in edges or in distance [1]. On this data set, DFS from the Entrance goes first towards `Reference`, then onwards, and produces long detours (Section 4.3). Like BFS, it ignores the `restricted` flag, and the observed DFS routes sometimes pass *through* `Restricted_Archive` on the way to an unrelated target.

3.3.3 A* Search

Purpose. A* finds a least-cost path using both the cost already accumulated and an estimate of the cost remaining [2], [1]. Each queue entry is ranked by the evaluation function

$$f(n) = g(n) + h(n) \tag{1}$$

where g(n) is the cost of the path from the start to node n and h(n) is the heuristic estimate of the cost from n to the goal. **Data structures.** A binary heap managed by `heapq` [6] holds tuples `(f, g, node, path)` and plays the role of the **open list**; a `visited` set plays the role of the **closed list**. **Steps.** A* pops the entry with the smallest f. If its node is the goal, A* returns the path, the cost g, the explored count and the closed list. If the node is already in the closed list, the entry is skipped. Otherwise the node is closed and counted, and for every neighbour that is neither in a closed section nor restricted-without-authorisation, A* computes the new cost, adds the heuristic and pushes the new entry. If the heap empties, A* returns no path and an infinite cost. The code is shown in Listing 8.

A worked trace. Table 11 traces A* from the Entrance to `AI_Section` in distance mode, computed by hand from the graph data and the heuristic. The result (path Entrance → CS_Section → AI_Section, cost 15, four nodes explored, closed list {Entrance, CS_Section, Reference, Math_Section}) agrees with the output of the program for this request.

Table 11: Hand trace of A* from Entrance to AI_Section (distance mode); h values are Euclidean distances to AI_Section at (4, 4)

Step	Node popped	g	f	Neighbours pushed as (node, g, f)
1	Entrance	0	0	CS_Section (10, 12.83); Math_Section (8, 14.32); Literature (12, 18.32); Reference (5, 13.49)
2	CS_Section	10	12.83	AI_Section (15, 15.00); Entrance (20, 25.66); Restricted_Archive skipped (restricted)
3	Reference	5	13.49	Entrance (10, 15.66); Math_Section (11, 17.32); Literature (13, 19.32)
4	Math_Section	8	14.32	Entrance (16, 21.66); Reference (14, 22.49); Physics_Section (14, 22.00); Restricted_Archive skipped
5	AI_Section	15	15.00	Goal reached: return path, cost 15, explored 4

The trace shows the heuristic at work: `CS_Section` has the lowest f because it lies in the direction of the goal, and `Literature` and `History` are never expanded. **Strengths and limitations.** A* explores fewer nodes than an uninformed method when the heuristic is informative, and it returns a least-cost path when the heuristic is admissible. Its weaknesses here are that the code returns an unordered closed list, ignores the open list in its output, and represents paths as full lists copied on every push, which is simple but memory-hungry on large graphs.

3.3.4 Heuristic Function and Path-Cost Calculation

The cost of a path is the sum of its edge costs. For a neighbour n′ reached from n over an edge of cost c(n, n′), the code computes

$$g(n') = g(n) + c(n, n') \tag{2}$$

where c is read from the `distance` field in distance mode or the `time` field in time mode. The heuristic is the straight-line distance between the node and the goal, computed from the stored coordinates [7]:

$$h(n) = \sqrt{((x_g - x_n)^2 + (y_g - y_n)^2)} \quad (3)$$

A heuristic is **admissible** if it never overestimates the true remaining cost [2], [1]:

$$h(n) \leq h^*(n) \quad (4)$$

where h*(n) is the true least cost from n to the goal. Whether the Euclidean heuristic is admissible depends on the data, because edge distances in the JSON file were typed by hand rather than computed from the coordinates. For this data set, the distance of each of the 12 edges was compared with the Euclidean distance between its end points, and **no edge was shorter than the straight line**. Straight-line distance therefore never exceeds the cost of any route in this graph, so the heuristic is admissible here, and because it also satisfies the triangle inequality along each edge it is consistent, which is what the closed-set version of A* used in the code needs to guarantee optimal paths [1]. This is a property of the supplied data and not of the code: an edge with a distance smaller than its straight-line length would break it.

In time mode the code sets h = 0 (`algorithms.py`, line 83). A* with a zero heuristic behaves like uniform-cost search, which is Dijkstra's algorithm with a priority queue [11], [3]. It is correct but explores more nodes. In the supplied data the `time` of every edge equals its `distance`, so time mode produces the same costs as distance mode.

3.3.5 Comparison of Search Algorithms

Table 12: Theoretical properties compared with behaviour in this project

Aspect	BFS	DFS	A*
Data structure	FIFO list of paths	LIFO list of paths	Binary heap (open list)
Uses edge costs	No	No	Yes (g and h)
Theory: optimal path	Fewest edges only	Not guaranteed	Least cost if h is admissible
Theory: complete	Yes (finite graph)	Yes with visited set	Yes (finite graph)
Restricted nodes	Ignored	Ignored	Respected
Closed sections	Respected (function level)	Respected (function level)	Respected (function level)
Return values	path, explored	path, explored	path, cost, explored, closed list
Works through the API	No (HTTP 500)	No (HTTP 500)	Yes

Table 13: Asymptotic behaviour for a graph with V nodes and E edges (textbook values [3], [1]; not measured here)

Measure	BFS	DFS	A*
Time	O(V + E) in standard form	O(V + E) in standard form	Depends on the heuristic; O((V + E) log V) with a heap in the best case
Space	O(V)	O(V) (depth-bounded in trees)	O(V) open and closed lists

> **Note.** The implementation stores a whole path in every queue entry, so its actual cost is higher than the standard bounds above. The bounds are quoted as theory; no timing measurements were made, because a nine-node graph is too small to say anything meaningful about running time.

3.4 Intelligent Agent and Multi-Book Planning

3.4.1 Goal-Based Search Behaviour

The planner is a goal-based agent in the sense of Russell and Norvig [1]. Its current state is its location; its goals are the shelves of the books in the collection; its action is a walk computed by A*; and its performance measure is the number of books collected within the limit. It does not learn or reason about the future beyond one book at a time. Figure 3 shows its control flow.

3.4.2 Priority-Based Book Collection

Before planning, the books are sorted with `sorted()` using a rank of 1 for High, 2 for Medium and 3 for Low (unknown values default to Low). Python's sort is stable, so books of equal priority keep the order in which they were added to the collection. The planner then handles the books in that order; the sorted order is a *processing* order, not a promise that every high-priority book is collected.

3.4.3 Greedy Nearest-Neighbor Route Planning

For each pending book, the planner runs A* from the current location to that book's shelf and, if the result is acceptable, extends the route and moves the current location to the shelf. The choice at each step is made without looking ahead, which makes the strategy **greedy**. Strictly, it is a priority-ordered greedy planner: the next book is chosen by priority, not by distance, and A* supplies the least-cost path only for that one leg. The planner is therefore **not** a solution to the travelling salesperson problem, which is NP-hard in general [3], [12], and the route it returns is not guaranteed to be the shortest route that collects the books. A book on the same shelf as the current location costs 0 and is picked up immediately.

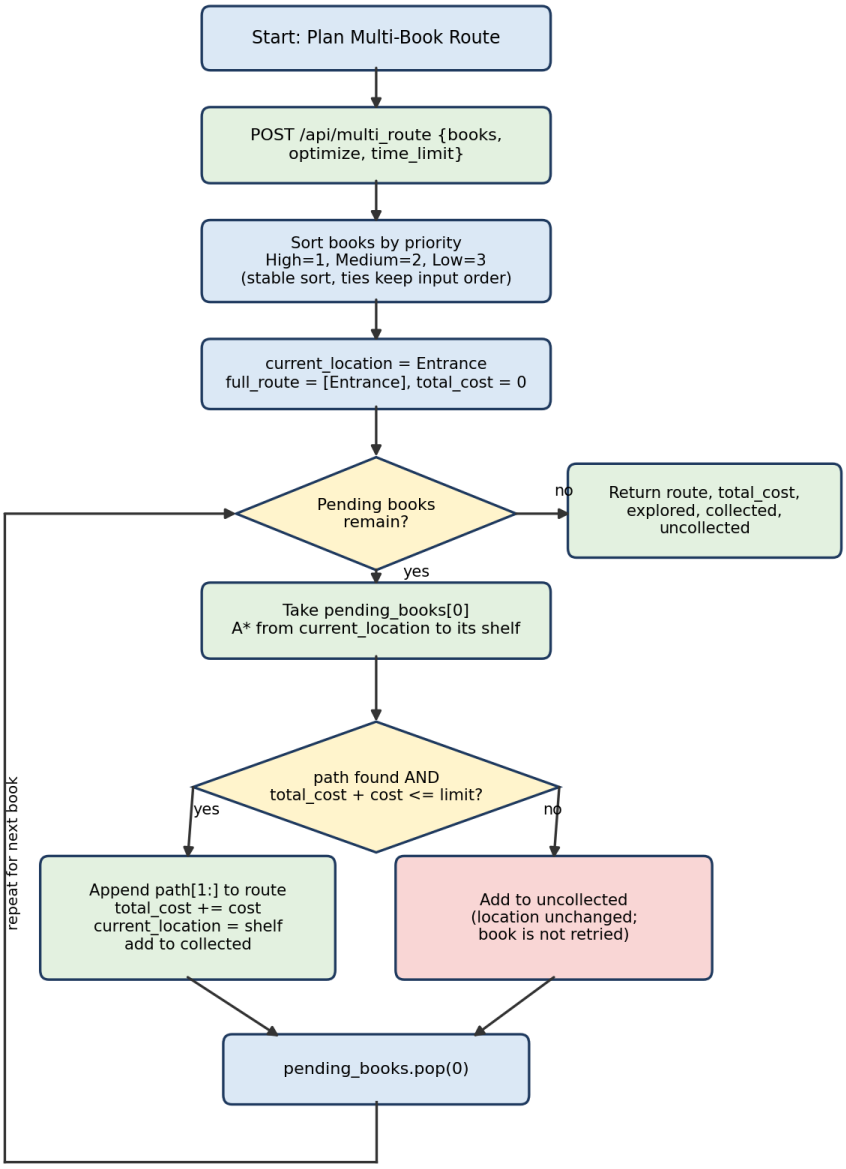


Figure 3: Flowchart of multi-book planning in plan_collection_route()

3.4.4 Time/Cost Limit Handling

A leg is accepted only if a path exists **and** `total_cost + cost <= time_limit`. The limit is measured in the same unit as the chosen optimisation mode (distance units or time units) and applies to the cumulative cost from the Entrance; the walk back to the Entrance is neither planned nor counted. A rejected book is added to the list of uncollected books, the current location is left unchanged, and the planner moves on to the next book. A rejected book is never retried. Two consequences were observed (Section 4.5): a lower-priority book that fits can be collected after a higher-priority book was rejected, and a small change in the limit can change which books are collected in a non-monotonic way. A restricted shelf is rejected in the same way, because A* returns no path when `authorized` is false.

3.5 Library Navigation and Collection Management

From the user's perspective there are two separate workflows. **Collection management** is a browser-only activity: *find*, *add* and *clear*. A found book is held in `currentFoundBook`; adding it appends it to `collectionList` unless it is

unavailable or already present. **Navigation** has two forms. Single-book navigation takes the shelf of the *last* book added and sends it to `/api/navigate` with the chosen algorithm and optimisation mode (Figure 4); the other books in the collection are ignored. Multi-book navigation sends the whole collection to `/api/multi_route` together with the limit.

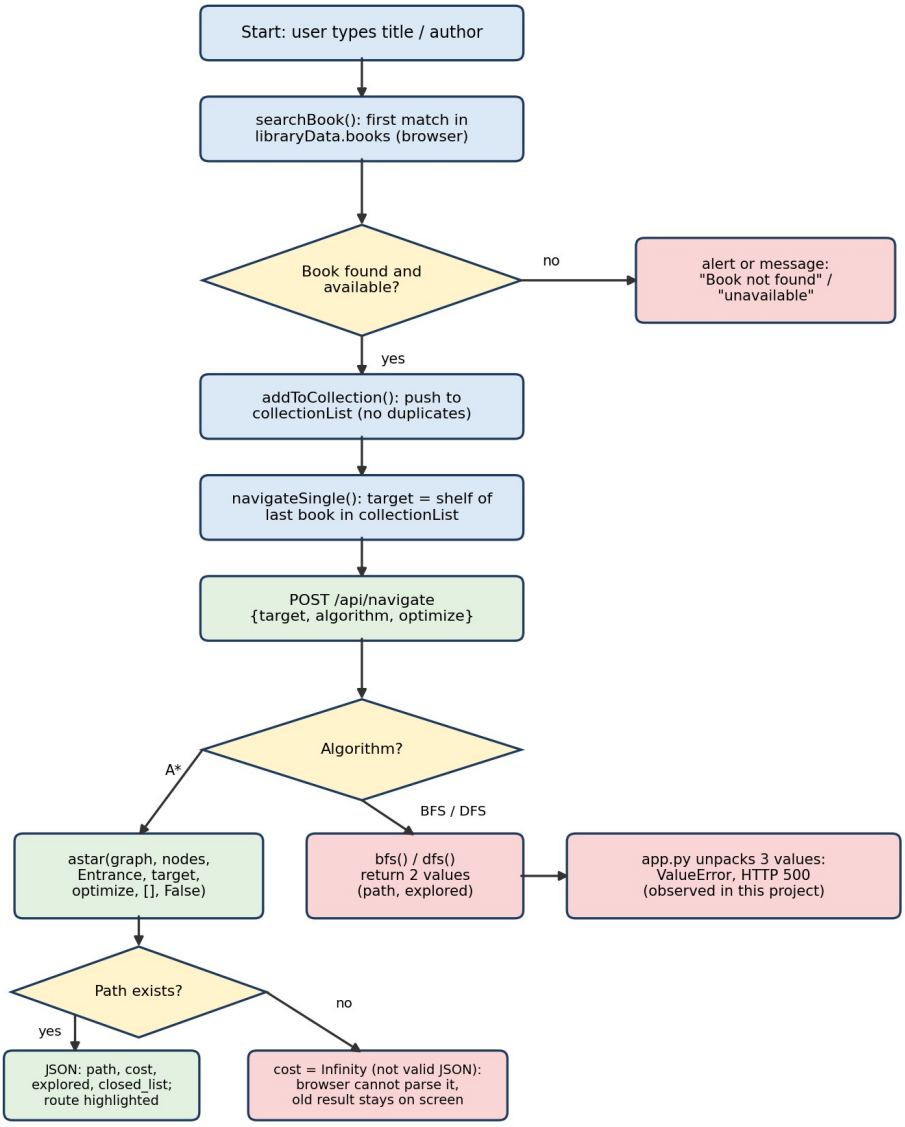


Figure 4: Flowchart of single-book navigation, including the failure paths observed during testing

3.6 Implementation

3.6.1 Backend Implementation

The back end consists of three Python files. `algorithms.py` (90 lines) contains `heuristic`, `bfs`, `dfs` and `astar`. `multi_book_agent.py` (41 lines) contains `plan_collection_route`. `app.py` (75 lines) loads the JSON file, rebuilds the graph, defines the four routes and starts Flask in debug mode. The back end performs no input validation: it trusts the target name, the algorithm name, the limit and the book objects sent by the browser. Section 3.7 reproduces and explains the important code.

3.6.2 Frontend Implementation

The front end is a single page (`templates/index.html`) styled by `static/style.css` and driven by `static/script.js` (248 lines). The page has a 350 px sidebar with a search box, the collection list, an algorithm selector, an optimisation selector, a numeric limit (default 100) and two action buttons, plus a graph panel and a Results panel. `script.js` keeps three pieces of state: `libraryData` (the data fetched at start-up), `currentFoundBook` and `collectionList`.

3.6.3 API Communication

The browser communicates with Flask through the `fetch` API [9] using JSON bodies. `navigateSingle()` posts `{target, algorithm, optimize}`, and `navigateMulti()` posts `{books, optimize, time_limit}`, where `books` is the array of complete book objects (so that the server can see each book's shelf and priority). The responses are described in Table 5. Neither function checks the HTTP status or handles a parse failure; if the response is not valid JSON, the awaited `res.json()` throws and the Results panel keeps whatever it showed before.

3.6.4 Graph Visualization

`drawGraph(pathArray)` builds Vis.js node and edge datasets [5]. Section nodes are drawn as boxes (light blue, with the restricted node in red); each book becomes a small yellow ellipse joined to its section by a dashed line; section edges are labelled with their distance. When a path is supplied, the nodes on the path are coloured green and the edges between consecutive path nodes are drawn thicker and green. Physics is switched off after the layout stabilises and the camera is fitted to the graph. The node positions on screen come from the stored coordinates.

3.6.5 Testing and Error Handling

`test_algorithm.py` defines five `unittest` cases [10] on a five-node test graph that is separate from the library data: one for BFS and four for A* (optimal cost, closed-section avoidance, restricted access denied and granted). There is no DFS test, no multi-book test and no API test. The results of running them are given in Section 4.7: **one passes and four fail with an error**, because the tests unpack three values from `astar`, which returns four.

Error handling in the application itself is minimal. The browser shows an alert when the user adds from an empty search, adds an unavailable book, adds a duplicate, or navigates with an empty collection. In the back end, an unreachable A* target is reported by returning `None` and an infinite cost, and the multi-book planner converts a failed leg into an "uncollected" entry. Other failure cases (unknown target, malformed limit, BFS and DFS requests) end in an unhandled server exception (HTTP 500).

3.7 Source Code

The excerpts below are copied unchanged from the submitted files; the numbers at the left are the line numbers in the original file. Boilerplate such as imports and the styling file is omitted.

3.7.1 Important Backend Code

Listing 2: Loading the data and rebuilding the graph (python-files/app.py, lines 7–22)

```
 7  app = Flask(__name__)
 8
 9  # Load the library data
10  base_dir = os.path.dirname(__file__)
11  json_path = os.path.join(base_dir, '..', 'JSON', 'library_data.json')
12  with open(json_path, 'r') as f:
13      library_data = json.load(f)
14
15  # Rebuild graph structure for algorithms
16  graph = {}
17  for node in library_data['nodes']:
18      graph[node] = {}
19  for edge in library_data['edges']:
20      u, v = edge['from'], edge['to']
21      graph[u][v] = {'distance': edge['distance'], 'time': edge['time']}
22      graph[v][u] = {'distance': edge['distance'], 'time': edge['time']}
```

Lines 10–13 locate `../JSON/library_data.json` relative to `app.py` and load it once at start-up. **Lines 16–18** create an empty adjacency dictionary for every node. **Lines 19–22** insert each edge twice, once in each direction, with its distance and time, which makes the graph undirected. The `nodes` dictionary from the JSON file is kept as `library_data['nodes']` and passed to A* for coordinates and restriction flags.

Listing 3: The /api/navigate route (python-files/app.py, lines 32–49)

```
32  @app.route('/api/navigate', methods=['POST'])
33  def navigate():
34      data = request.json
35      target = data.get('target')
36      algo = data.get('algorithm', 'A*')
37      optimize = data.get('optimize', 'distance')
38      start = "Entrance"
39
40      if algo == "BFS":
41          path, explored, closed_list = bfs(graph, start, target, [], False)
42          cost = len(path) - 1 if path else 0
43      elif algo == "DFS":
44          path, explored, closed_list = dfs(graph, start, target, [], False)
45          cost = len(path) - 1 if path else 0
46      else:
47          path, cost, explored, closed_list = astar(graph, library_data['nodes'], start, target, optimize,
    [], False)
48
49      return jsonify({"path": path, "cost": cost, "explored": explored, "closed_list": closed_list})
```

Lines 34–38 read the request fields and fix the start at the Entrance. **Lines 40–45** handle BFS and DFS: each function is called with an empty closed-section list and `authorized=False`, and the result is unpacked into **three** variables, although `bfs` and `dfs` return only two values (path and explored count). Python therefore raises a `ValueError` for every BFS or DFS request, and Flask answers with HTTP 500. The intended cost for BFS and DFS is the number of edges (`len(path) - 1`), not a weighted distance. **Lines 46–47** call A* with the same fixed arguments, so the interface can never pass closed sections or authorisation. The response on line 49 contains the path, cost, explored count and closed list.

Listing 4: The /api/multi_route route (python-files/app.py, lines 51–71)

```
51  @app.route('/api/multi_route', methods=['POST'])
52  def multi_route():
53      data = request.json
54      books = data.get('books', [])
55      time_limit = int(data.get('time_limit', 9999))
56      optimize = data.get('optimize', 'distance')
57      authorized = data.get('authorized', False)
58
59      start = "Entrance"
60
61      path, cost, explored, collected, uncollected = plan_collection_route(
62          graph, library_data['nodes'], start, books, optimize, time_limit, authorized
63      )
64
65      return jsonify({
66          "path": path,
67          "distance": cost,
68          "explored": explored,
69          "collected": [b['title'] for b in collected],
70          "uncollected": [b['title'] for b in uncollected]
71      })
```

Line 55 converts the limit with `int(...)`, which fails with a `TypeError` if the browser sends `null` (what `JSON.stringify` produces for `NaN` when the limit box is empty). Line 57 reads an optional `authorized` flag that the supplied interface never sends. Lines 61–71 call the planner and return the route, the cost (under the key `distance`), the explored count and the titles of collected and uncollected books.

3.7.2 Search Algorithm Implementations

Listing 5: The Euclidean heuristic (python-files/algorithms.py, lines 4–8)

```
4   def heuristic(node_a, node_b, nodes_data):
5       """Calculates the Euclidean (straight-line) distance between two nodes."""
6       x1, y1 = nodes_data[node_a]['x'], nodes_data[node_a]['y']
7       x2, y2 = nodes_data[node_b]['x'], nodes_data[node_b]['y']
8       return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
```

The function reads the coordinates of both nodes and returns the straight-line distance, Equation (3).

Listing 6: Breadth-First Search (python-files/algorithms.py, lines 10–31)

```
10  def bfs(graph, start, goal, closed_sections, authorized):
11      """Breadth-First Search (Finds path with fewest edges)."""
12      queue = [[start]]
13      visited = set()
14      explored_count = 0
15
16      while queue:
17          path = queue.pop(0)
18          node = path[-1]
19
20          if node == goal:
21              return path, explored_count
22
23          if node not in visited:
24              visited.add(node)
25              explored_count += 1
26              for neighbor in graph.get(node, {}):
27                  if neighbor not in closed_sections:
28                      new_path = list(path)
29                      new_path.append(neighbor)
30                      queue.append(new_path)
31      return None, explored_count
```

Line 12 starts the queue with a one-node path. **Lines 16–18** dequeue the oldest path (`pop(0)`) and look at its last node. **Lines 20–21** return when the goal is reached. **Lines 23–25** expand a node only the first time it is dequeued and count it. **Lines 26–30** enqueue a copy of the path extended by each neighbour that is not in `closed_sections`. **Line 31** returns `None` if the queue empties. Notice that the `authorized` parameter and the `restricted` flag are never consulted.

Listing 7: Depth-First Search (python-files/algorithms.py, lines 33–54)

```
33  def dfs(graph, start, goal, closed_sections, authorized):
34      """Depth-First Search (Explores deep before backtracking)."""
35      stack = [[start]]
36      visited = set()
37      explored_count = 0
38
39      while stack:
40          path = stack.pop()
41          node = path[-1]
42
43          if node == goal:
44              return path, explored_count
45
46          if node not in visited:
47              visited.add(node)
48              explored_count += 1
49              for neighbor in graph.get(node, {}):
50                  if neighbor not in closed_sections:
51                      new_path = list(path)
52                      new_path.append(neighbor)
53                      stack.append(new_path)
54      return None, explored_count
```

The structure is identical to BFS except that line 40 removes the **newest** path (`stack.pop()`), so the search dives along the most recently added neighbour before backtracking.

Listing 8: A* Search (python-files/algorithms.py, lines 56–90)

```
56  def astar(graph, nodes_data, start, goal, optimize_for='distance', closed_sections=[], authorized=False):
57      """A* Search using f(n) = g(n) + h(n)."""
58      pq = [(0, 0, start, [start])] # This acts as our Open List
59      visited = set()               # This acts as our Closed List
60      explored_count = 0
61
62      while pq:
63          f_cost, g_cost, current, path = heapq.heappop(pq)
64
65          if current == goal:
66              # We return the path, cost, explored count, and the closed list
67              return path, g_cost, explored_count, list(visited)
68
69          if current in visited:
70              continue
71
72          visited.add(current)
73          explored_count += 1
74
75          for neighbor, costs in graph.get(current, {}).items():
76              if neighbor in closed_sections:
77                  continue
78              if nodes_data[neighbor].get('restricted', False) and not authorized:
79                  continue
80
81              edge_cost = costs[optimize_for]
82              new_g = g_cost + edge_cost
83              h_cost = heuristic(neighbor, goal, nodes_data) if optimize_for == 'distance' else 0
84              new_f = new_g + h_cost
85
86              new_path = list(path)
87              new_path.append(neighbor)
88              heapq.heappush(pq, (new_f, new_g, neighbor, new_path))
89
90      return None, float('inf'), explored_count, list(visited)
```

Lines 58–60 initialise the open list as a heap with the start entry, an empty closed set and the counter. **Lines 63–67** pop the entry with the lowest f and return if it is the goal. **Lines 69–73** skip nodes already closed and otherwise close and count the node. **Lines 75–79** loop over neighbours, skipping closed sections and restricted nodes when `authorized` is false; because the check is applied to every neighbour, a restricted *goal* is also unreachable without authorisation. **Lines 81–84** compute the edge cost, the new g, the heuristic (zero in time mode) and the new f. **Lines 86–88** push the extended path. **Line 90** returns no path, infinite cost and the explored count. The mutable default `closed_sections=[]` on line 56 is harmless here because the list is never modified.

3.7.3 Multi-Book Planning Implementation

Listing 9: Priority-ordered multi-book planner (python-files/multi_book_agent.py, lines 3–41)

```
3  def plan_collection_route(graph, nodes_data, start_node, target_books, optimize_for='distance',
   time_limit=float('inf'), authorized=False):
4      current_location = start_node
5
6      # ADVANCED FEATURE: Sort books by Priority (High first, then Medium, then Low)
7      priority_map = {"High": 1, "Medium": 2, "Low": 3}
8      pending_books = sorted(target_books, key=lambda b: priority_map.get(b.get('priority', 'Low'), 3))
9
10     full_route = [start_node]
11     total_cost = 0
12     total_explored = 0
13     collected_books = []
14     uncollected_books = []
15
16     while pending_books:
17         best_next_book = pending_books[0] # Take highest priority book
18         target_shelf = best_next_book['shelf']
19
20         path, cost, explored, closed_list = astar(
21             graph, nodes_data, current_location, target_shelf,
22             optimize_for=optimize_for, authorized=authorized
23         )
24
25         # ADVANCED FEATURE: Check Time/Cost Limit
26         if path and (total_cost + cost <= time_limit):
27             if len(full_route) > 0 and path[0] == full_route[-1]:
28                 full_route.extend(path[1:])
29             else:
30                 full_route.extend(path)
31
32             total_cost += cost
33             total_explored += explored
34             current_location = target_shelf
35             collected_books.append(best_next_book)
36         else:
37             uncollected_books.append(best_next_book)
38
39         pending_books.pop(0)
40
41     return full_route, total_cost, total_explored, collected_books, uncollected_books
```

Lines 7–8 sort the books by priority rank; the sort is stable. **Lines 10–14** initialise the route (starting at the Entrance), the running cost and counters, and the two result lists. **Lines 16–23** take the first pending book and run A* from the current location to its shelf. **Line 26** is the limit test: the leg must exist and must keep the cumulative cost within `time_limit`. **Lines 27–30** append the new leg to the route (skipping its first node, which duplicates the current end of the route). **Lines 32–35** update the cost, explored count, location and collected list. **Lines 36–37** record a rejected book without moving. **Line 39** removes the book from the pending list. Note that `total_explored` accumulates only for accepted legs, so the figure excludes the search effort spent on rejected books.

3.7.4 Frontend and API Integration

Listing 10: Searching and collecting books in the browser (python-files/static/script.js, lines 14–60)

```
14  function searchBook() {
15      const query = document.getElementById('searchInput').value.toLowerCase();
16      currentFoundBook = null;
17
18      for (let book of libraryData.books) {
19          if (book.title.toLowerCase().includes(query) || book.author.toLowerCase().includes(query)) {
20
21              // Check if the book is already in the collection list
22              const isAlreadyAdded = collectionList.some(b => b.id === book.id);
23
24              // Determine the status text to display
25              let status = "Available";
26              if (!book.available) {
27                  status = "Unavailable";
28              } else if (isAlreadyAdded) {
29                  status = "Unavailable (Already in Collection)";
30              }
31
32              document.getElementById('searchResult').innerHTML = `Found: <b>${book.title}</b><br>Shelf: $
    {book.shelf} (${status})`;
33
34              // Store the book temporarily, but force it to be "unavailable" if already added
35              currentFoundBook = {
36                  ...book,
37                  available: book.available && !isAlreadyAdded
38              };
39
40              return;
41          }
42      }
43      document.getElementById('searchResult').innerText = "Book not found.";
44  }
45
46  function addToCollection() {
47      if (!currentFoundBook) return alert("Search for a book first!");
48      if (!currentFoundBook.available) return alert("Book is unavailable.");
49
50      const isAlreadyAdded = collectionList.some(book => book.id === currentFoundBook.id);
51      if (isAlreadyAdded) {
52          return alert("You have already added this book to your collection!");
53      }
54
55      collectionList.push(currentFoundBook);
56      updateCollectionUI();
57      currentFoundBook = null;
58      document.getElementById('searchResult').innerText = "";
59      document.getElementById('searchInput').value = "";
60  }
```

`searchBook()` lowercases the query and scans `libraryData.books` for the **first** book whose title or author contains it (lines 18–19); later matches are not shown. A book already in the collection is displayed as “Unavailable (Already in Collection)” and the copy stored in `currentFoundBook` is marked unavailable (lines 22–38). `addToCollection()` rejects an empty search, an unavailable book and a duplicate with alerts (lines 47–53) before appending the book and updating the list (lines 55–59).

Listing 11: Single-book navigation in the browser (python-files/static/script.js, lines 180–207)

```
180 async function navigateSingle() {
181     if (collectionList.length === 0) return alert("Add a book to collection first.");
182 
183     const target = collectionList[collectionList.length - 1].shelf;
184     const algo = document.getElementById('algoSelect').value;
185 
186     // Grab the optimization choice from HTML
187     const optimizeDropdown = document.getElementById('optimizeSelect');
188     const optimizeChoice = optimizeDropdown ? optimizeDropdown.value : 'distance';
189 
190     const res = await fetch('/api/navigate', {
191         method: 'POST',
192         headers: { 'Content-Type': 'application/json' },
193         body: JSON.stringify({
194             target: target,
195             algorithm: algo,
196             optimize: optimizeChoice
197         })
198     });
199 
200     const data = await res.json();
201     if (data.path) {
202         document.getElementById('resultText').innerHTML = `<b>Cost:</b> ${data.cost}<br><b>Explored
    Nodes:</b> ${data.explored}<br><b>Closed List:</b> ${data.closed_list.join(', ')}<br><b>Route:</b> $
    {data.path.join(' ➔ ')}`;
203         drawGraph(data.path);
204     } else {
205         document.getElementById('resultText').innerText = "No path found.";
206     }
207 }
```

Line 183 chooses the shelf of the last book in the collection as the target. Lines 190–198 post the request. Line 200 parses the response as JSON; if the server returned HTTP 500 (an HTML error page) or `Infinity`, this line throws and nothing after it runs. Lines 201–203 print the cost and draw the path when a path exists.

Listing 12: Multi-book navigation in the browser (python-files/static/script.js, lines 209–248)

```
209 async function navigateMulti() {
210     if (collectionList.length === 0) return alert("Collection is empty.");
211 
212     // Grab the actual book objects so the backend knows their priorities
213     const booksData = collectionList.map(b => b);
214 
215     // Grab the advanced feature inputs from HTML
216     const optimizeDropdown = document.getElementById('optimizeSelect');
217     const optimizeChoice = optimizeDropdown ? optimizeDropdown.value : 'distance';
218 
219     const timeLimitInput = document.getElementById('timeLimit');
220     const timeLimitValue = timeLimitInput ? parseInt(timeLimitInput.value) : 9999;
221 
222     const res = await fetch('/api/multi_route', {
223         method: 'POST',
224         headers: { 'Content-Type': 'application/json' },
225         body: JSON.stringify({
226             books: booksData,
227             optimize: optimizeChoice,
228             time_limit: timeLimitValue
229         })
230     });
231 
232     const data = await res.json();
233     if (data.path) {
234         let msg = `<b>Distance/Time Used:</b> ${data.distance}<br><b>Explored:</b> ${data.explored}
    <br>`;
235         msg += `<b>Collected:</b> ${data.collected.join(', ')}<br>`;
236 
237         if (data.uncollected && data.uncollected.length > 0) {
238             msg += `<span style="color:red"><b>Out of Time! Missed:</b> ${data.uncollected.join(', ')}
    </span><br>`;
239         }
240 
241         msg += `<b>Route:</b> ${data.path.join(' ➔ ')}`;
242 
243         document.getElementById('resultText').innerHTML = msg;
244         drawGraph(data.path);
245     } else {
246         document.getElementById('resultText').innerText = "Failed to generate route.";
247     }
248 }
```

Line 220 reads the limit with `parseInt`; an empty box gives `NaN`, which is serialised as `null`. Lines 222–230 post the whole collection. Lines 233–244 build the Results text: the cost, the explored count, the collected titles, a red “Out of Time! Missed:” line for uncollected books (which is also shown for books missed because of a restriction, so the label is not always accurate) and the route, and then call `drawGraph`.

3.7.5 Unit Tests

Listing 13: The submitted unit tests (python-files/test_algorithm.py, lines 22–50)

```
22      def test_bfs_finds_path(self):
23          path, _ = bfs(self.graph, "Entrance", "AI", closed_sections=[], authorized=False)
24          self.assertIsNotNone(path)
25          self.assertEqual(path[0], "Entrance")
26          self.assertEqual(path[-1], "AI")
27
28      def test_astar_finds_optimal_path(self):
29          # A* should prefer Entrance -> CS -> AI (10+5=15) over Entrance -> Math -> AI (5+10=15)
30          # depending on heuristic, but both are valid. Let's test cost.
31          path, cost, _ = astar(self.graph, self.nodes_data, "Entrance", "AI", 'distance')
32          self.assertEqual(cost, 15)
33          self.assertEqual(path[-1], "AI")
34
35      def test_closed_sections_avoided(self):
36          # Close CS. The only way to AI is through Math.
37          path, cost, _ = astar(self.graph, self.nodes_data, "Entrance", "AI", 'distance',
    closed_sections=["CS"])
38          self.assertIn("Math", path)
39          self.assertNotIn("CS", path)
40
41      def test_restricted_access_denied(self):
42          # Unauthorized access to Restricted section should return None
43          path, cost, _ = astar(self.graph, self.nodes_data, "Entrance", "Restricted", 'distance',
    authorized=False)
44          self.assertIsNone(path)
45
46      def test_restricted_access_granted(self):
47          # Authorized access should find a path
48          path, cost, _ = astar(self.graph, self.nodes_data, "Entrance", "Restricted", 'distance',
    authorized=True)
49          self.assertIsNotNone(path)
50          self.assertEqual(path[-1], "Restricted")
```

The BFS test (lines 22–26) unpacks two values and matches the function. The four A* tests (lines 28–50) unpack **three** values, but `astar` returns four, so each raises `ValueError: too many values to unpack (expected 3)`. The intended assertions are sensible: optimal cost 15, avoidance of a closed section, denial of a restricted goal and success when authorised. Section 4.7 records both the actual results and a supplementary check of those intentions.

3.8 Output Screenshots

All screenshots below are genuine captures of the submitted application, taken with a headless Chromium browser at a viewport of 1400 × 1060 pixels while the Flask server was running (Section 4.1). They were taken by driving the real page; no image was edited or composed. Screenshots that show a failure are included deliberately, because they are real behaviour of the delivered code.

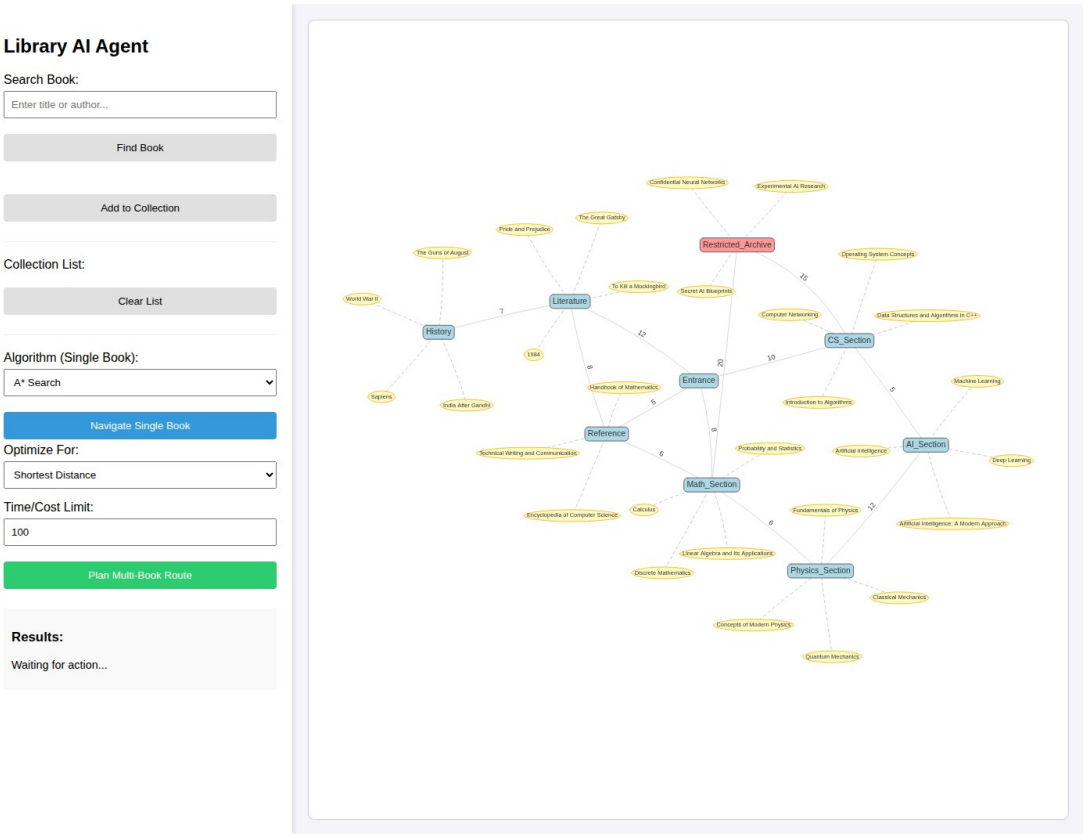


Figure 5: Main interface at start-up: controls in the sidebar and the library graph with section nodes, book nodes and edge distances

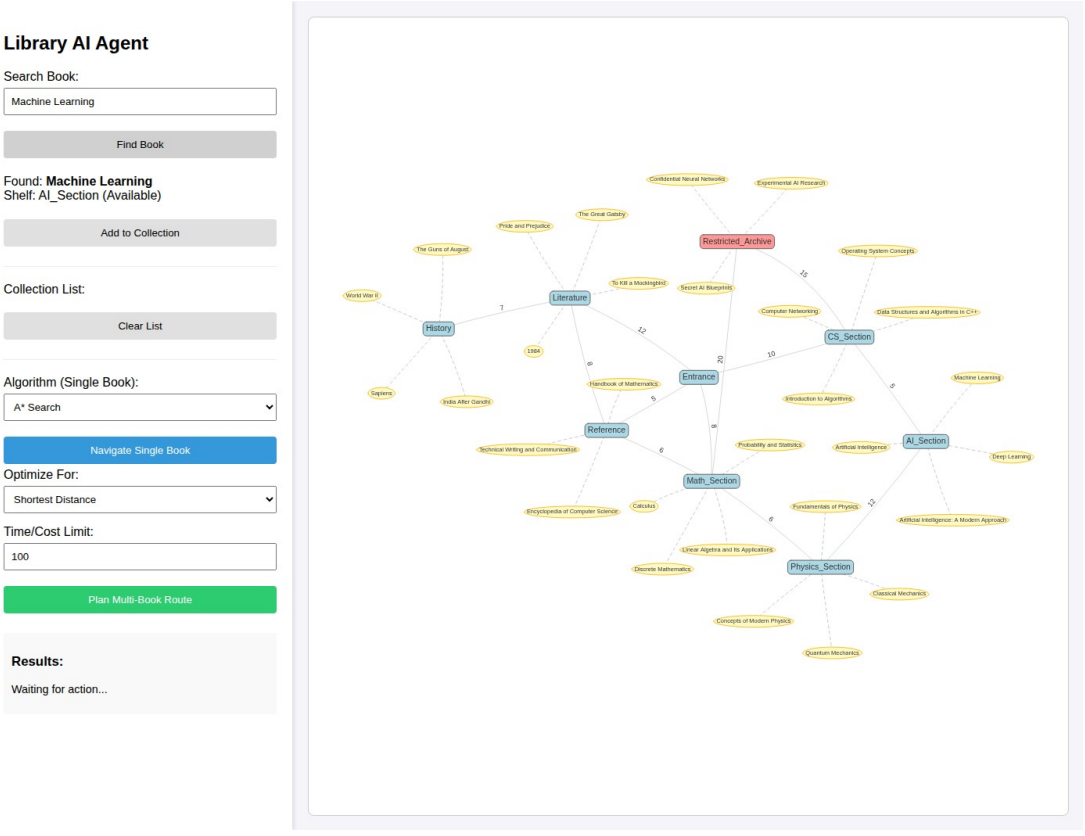


Figure 6: Book search result for "Machine Learning" showing shelf and availability

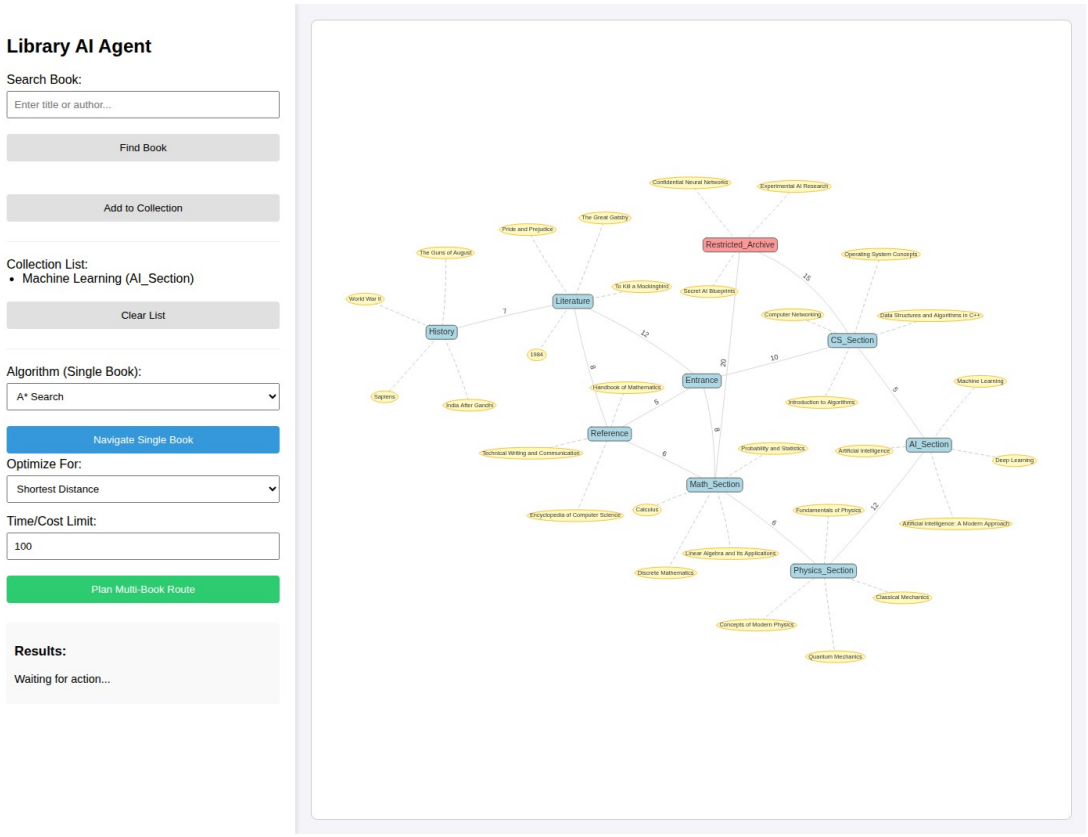


Figure 7: The book added to the collection list

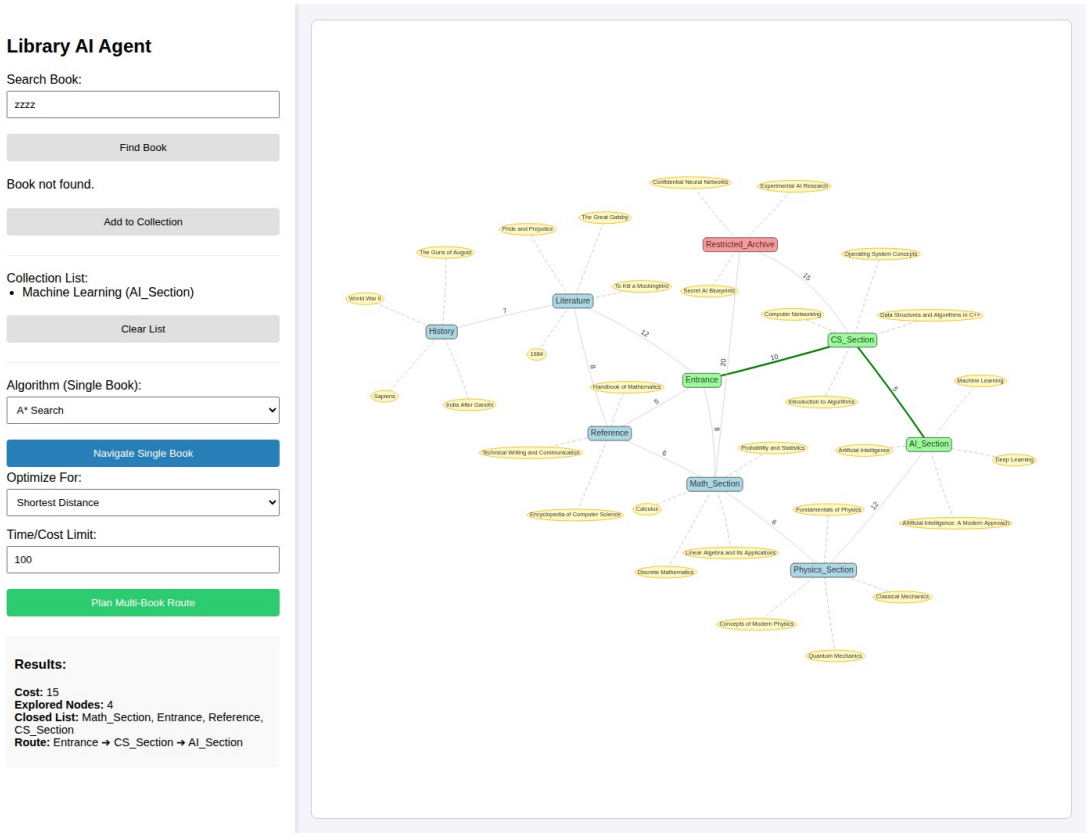


Figure 8: Single-book navigation to AI_Section with A* (cost 15, four nodes explored)

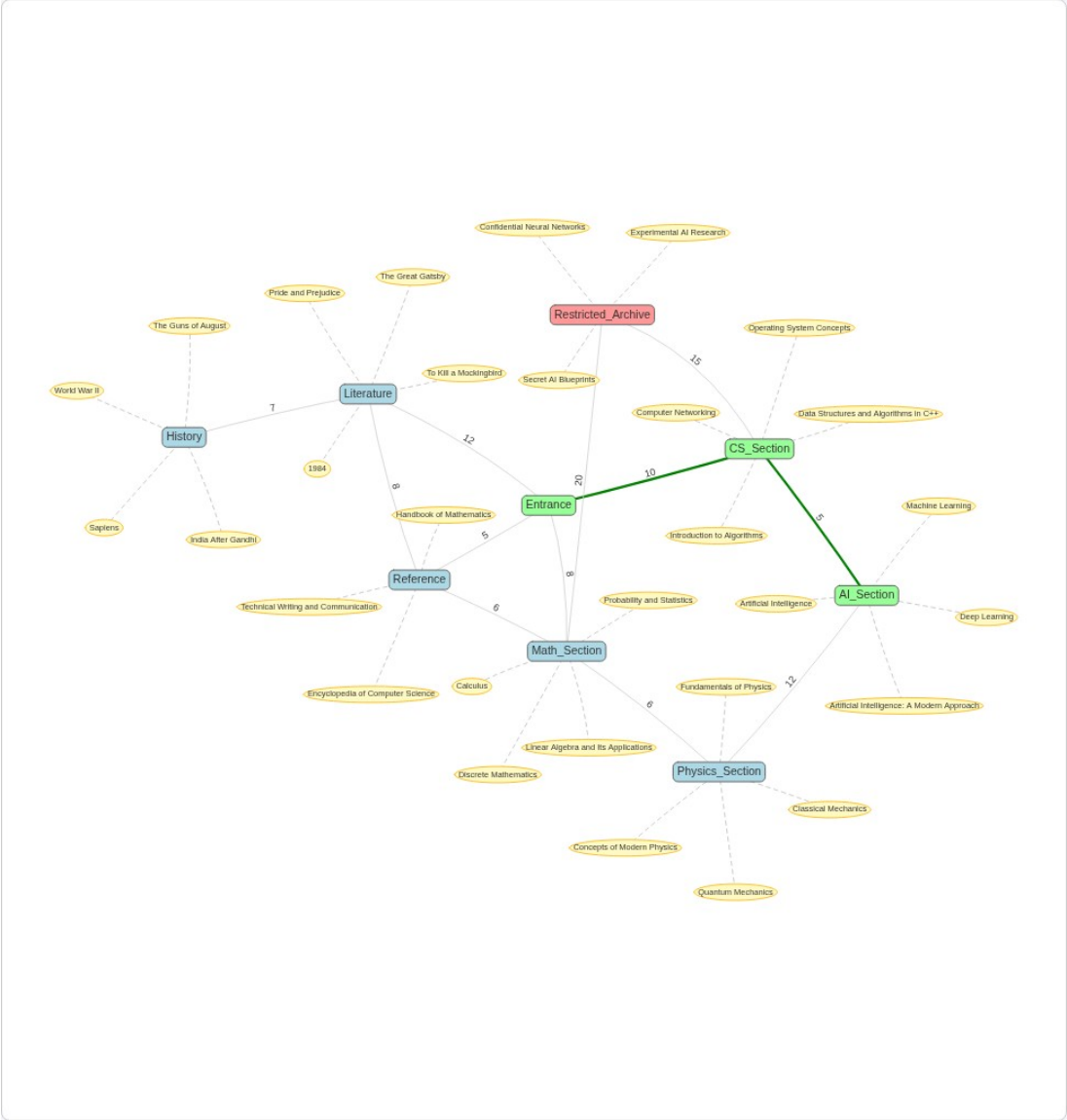


Figure 9: Graph panel with the A* route Entrance → CS_Section → AI_Section highlighted in green

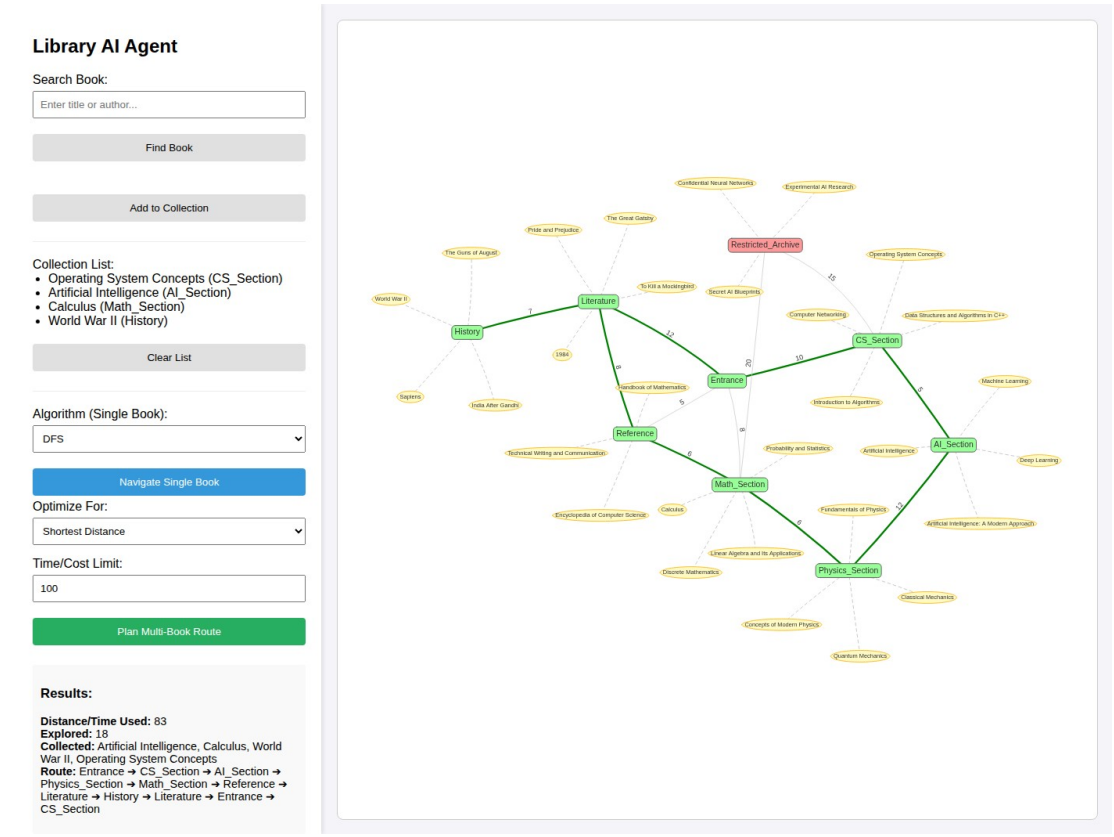


Figure 10: Multi-book plan for four books with a limit of 100 (all collected, cost 83)

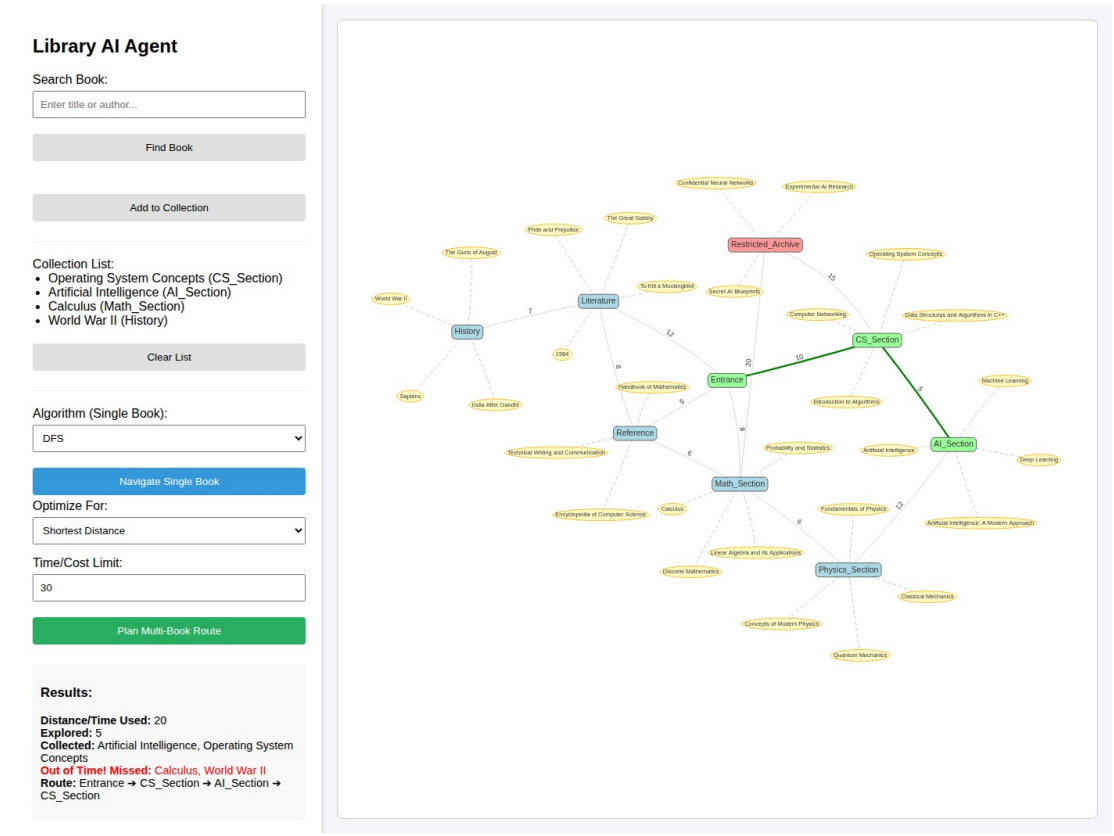


Figure 11: The same collection with a limit of 30: two books are reported as missed

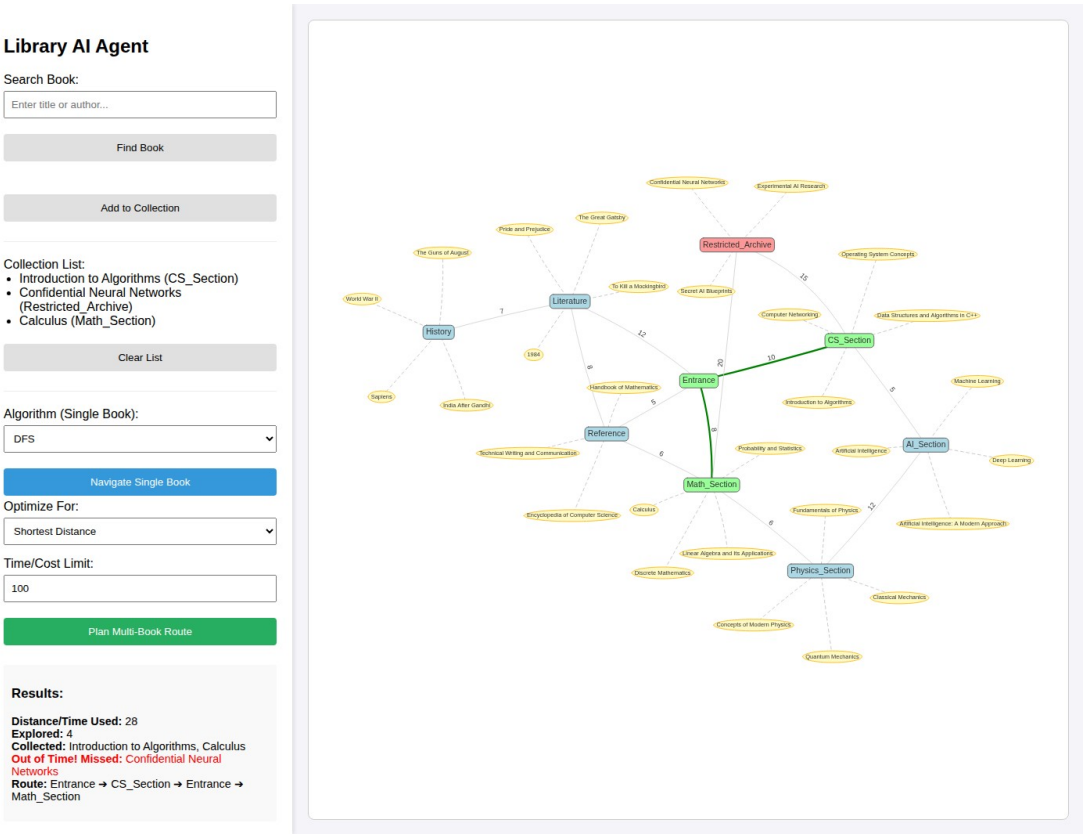


Figure 12: Collection containing a restricted-shelf book: the restricted book is missed because no authorisation is sent

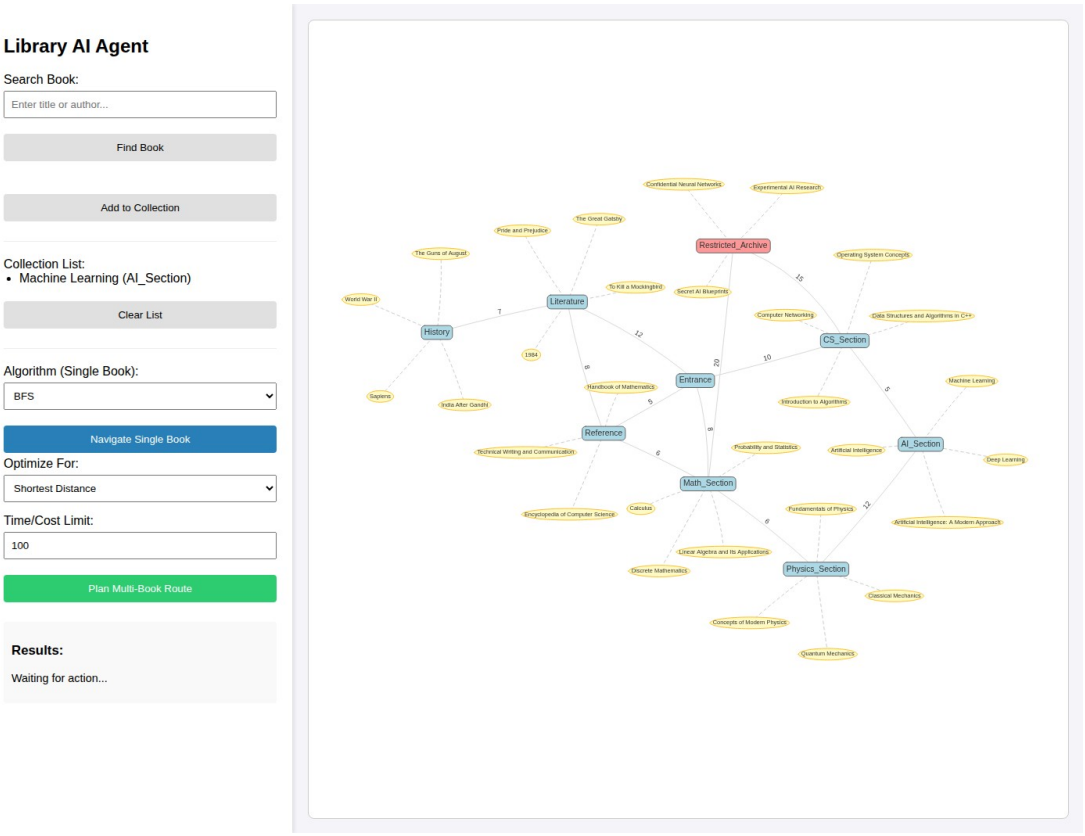


Figure 13: BFS selected for single-book navigation: the server answers HTTP 500 and the panel remains at "Waiting for action..."

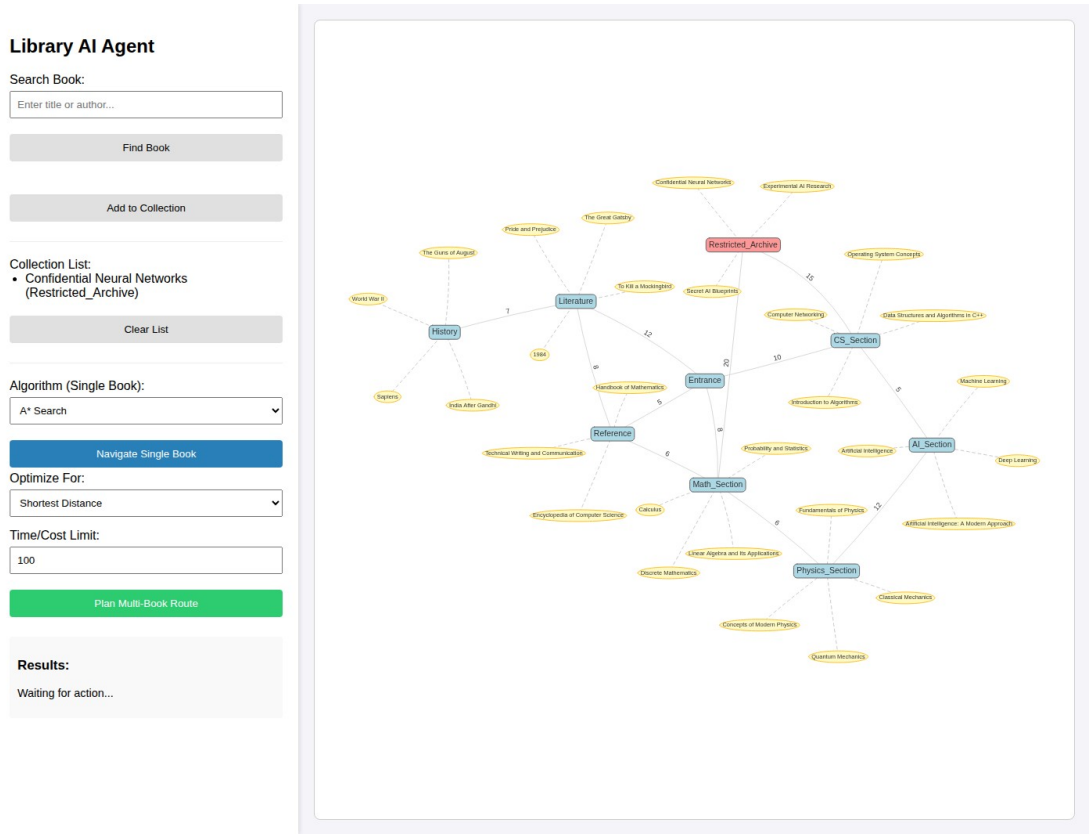


Figure 14: A* navigation to the restricted archive: the response contains Infinity, the browser cannot parse it and the panel does not change

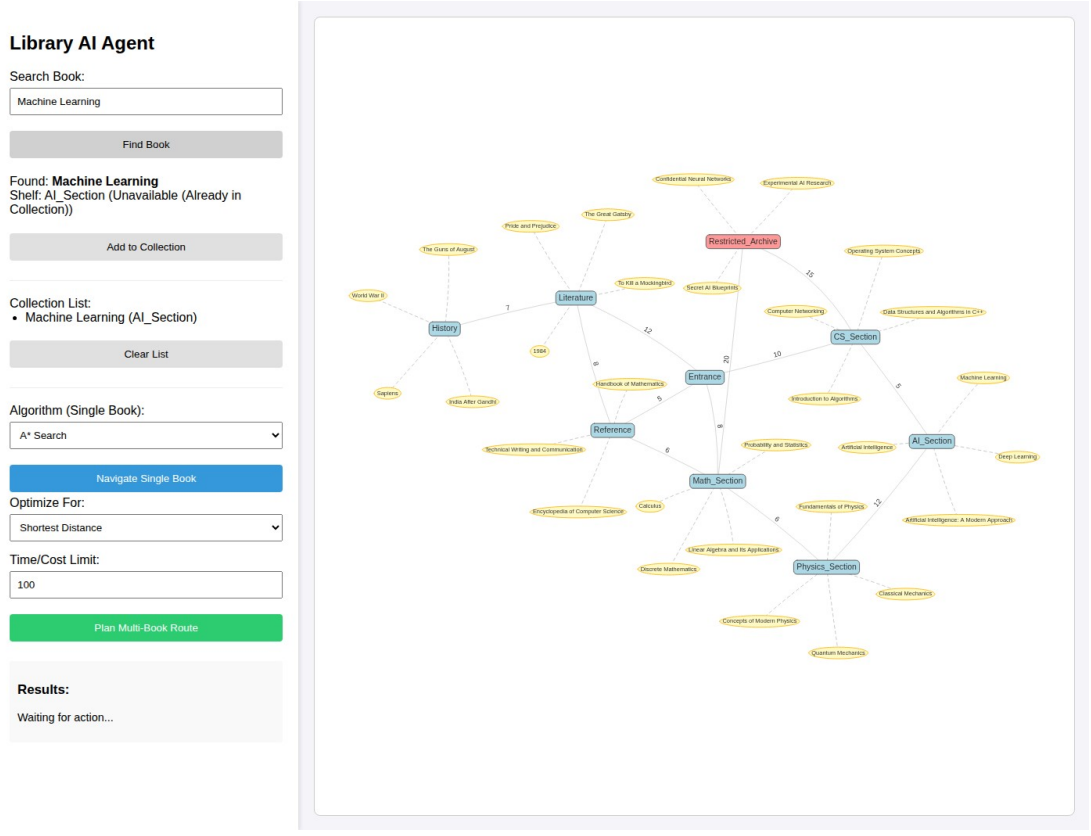


Figure 15: Searching for a book already in the collection: shown as unavailable

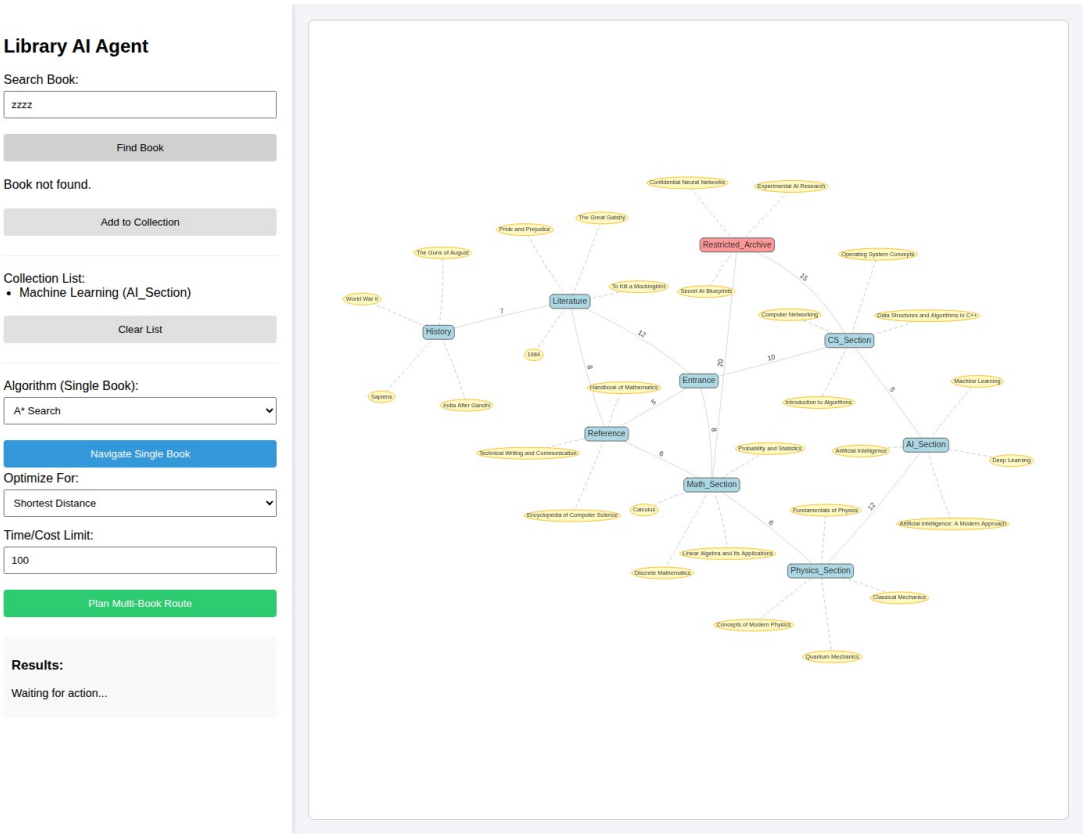


Figure 16: Search for a title that does not exist

The alert dialogs shown by the application (empty collection, duplicate or unavailable book) are native browser dialogs and are not captured in page screenshots; their text was recorded from the run instead (Section 4.8).

4 Result Analysis

This chapter reports what was observed when the submitted project was run. Every number comes from executing the submitted code on the supplied data; nothing was estimated. Where a result differs from the README or from a theoretical expectation, the difference is reported plainly. No execution times are given, because the graph has only nine nodes and no timing experiment was performed.

4.1 Test Environment

Table 14: Environment and method used for the experiments

Item	Details
Date of runs	27 October 2026
Operating system / hardware	Linux container (cloud sandbox); no hardware benchmark was taken.
Python / Flask	Python 3.12.3; Flask 3.1.3 (the README pins Flask 3.0.0, which was not used).
Browser	Headless Chromium driven by Playwright for Python 1.56.0 [13]; viewport 1400 × 1060 px.
Vis.js	The page normally loads Vis.js from unpkg, which was not reachable from the sandbox. A scratch copy of the project used a local copy of vis-network 10.1.4, with only the `<script>` source changed. The submitted files were not modified.
Server start	The scratch copy was started with `app.run(port=5055, debug=False)` instead of `python app.py`; the routes and logic are the submitted ones.
Three ways of testing	(1) Calling the Python functions directly with the real graph; (2) sending requests through the Flask test client; (3) operating the real page in the browser.
Reported values	`explored` is the counter returned by each function (nodes expanded, excluding the goal). For BFS and DFS the program returns no weighted cost, so "distance" for them was computed by summing the graph's edge distances along the returned path.

4.2 Single-Book Navigation Scenarios

Valid book, route exists. Searching for "Machine Learning" found a book on the shelf `AI_Section`, available; after adding it, A* returned the path Entrance → CS_Section → AI_Section with cost 15 and four explored nodes (Figure 6, Figure 7, Figure 8, Figure 9). The closed list was displayed in an arbitrary order (it is built from a Python set), and it varies from run to run.

Target unreachable. For the restricted archive without authorisation, A* returned no path, an infinite cost and eight explored nodes when called directly. Through the web interface the server returned HTTP 200 with the body `"cost":Infinity,"path":null`. `Infinity` is not valid JSON [8]; the browser reported a parse error and the Results panel did not change (Figure 14). The intended message "No path found." in `script.js` is therefore never shown for this case.

Unavailable book and duplicates. All 30 books are available in the supplied data, so the unavailable-book message could not be triggered with a genuine record. Searching for a book already in the collection displayed "Unavailable (Already in Collection)" (Figure 15), and pressing "Add to Collection" then produced the alert "Book is unavailable." The more specific alert "You have already added this book" is not reachable through the interface, because the search step has already marked the book unavailable. A search with no match showed "Book not found." (Figure 16).

Single-book scope. Single-book navigation uses only the *last* book added; with several books in the collection the others are ignored.

Table 15: Single-book results from the Entrance, distance mode, called directly (BFS and DFS: edges / weighted distance / explored; A*: cost / explored)

Target	BFS	DFS	A*	A* in time mode (explored)	A* authorised
CS_Section	1 / 10 / 1	4 / 46 / 6	10 / 1	3	10 / 1
AI_Section	2 / 15 / 5	5 / 51 / 7	15 / 4	6	15 / 4
Math_Section	1 / 8 / 2	2 / 11 / 4	8 / 1	2	8 / 1
Physics_Section	2 / 14 / 7	6 / 63 / 8	14 / 3	5	14 / 3
Literature	1 / 12 / 3	2 / 13 / 2	12 / 2	4	12 / 2
History	2 / 19 / 8	3 / 20 / 3	19 / 5	7	19 / 5
Reference	1 / 5 / 4	1 / 5 / 1	5 / 1	1	5 / 1
Restricted_Archive	2 / 25 / 6 (ignores restriction)	3 / 31 / 5 (ignores restriction)	no path (∞) / 8	8 (no path)	25 / 7

4.3 BFS, DFS, and A* Behaviour

Through the application. Selecting BFS or DFS in the browser always failed with HTTP 500 (`ValueError: not enough values to unpack (expected 3, got 2)`, `app.py` lines 41 and 44), and the Results panel remained at "Waiting for action..." (Figure 13). Only A* could be demonstrated through the interface. The comparison below therefore uses direct function calls on the same graph.

Direct comparison (Table 15). For the seven reachable, unrestricted targets A* found the least-cost routes, with explored counts between 1 and 5. In these runs BFS returned a route with the same weighted distance as A*, but it explored more nodes for `AI_Section` (5 against 4), `Physics_Section` (7 against 3), `History` (8 against 5) and `Reference` (4 against 1). DFS returned the same distance as A* only for `Reference`. The other DFS routes were longer: for `AI_Section` it returned the six-node route Entrance → Reference → Math_Section → Restricted_Archive → CS_Section → AI_Section with distance 51 against A*'s 15, and for `Physics_Section` it returned a distance of 63 against 14. Notice that these DFS routes pass through the restricted archive although the call used `authorized=False`.

Why BFS matched A* from the Entrance. BFS minimises edges, not distance. From the Entrance, the fewest-edge route happens to coincide with the least-distance route for every target in this graph. That is a property of this data and start node, not a general guarantee. To expose the difference, BFS and A* were compared for **all 72 ordered pairs of nodes** (called directly, with A* authorised so that all nodes could be used; BFS and DFS ignore restrictions in any case). Table 16 gives the summary and Table 17 lists some of the 12 pairs for which BFS's route was longer than the A* route.

Table 16: Comparison over all 72 ordered start/goal pairs (supplementary experiment, direct calls)

Measure	BFS	DFS	A*
Pairs whose route distance equals the A* distance	60 of 72	23 of 72	72 of 72 (reference)
Average nodes explored per pair	4.50	4.50	3.49

Table 17: Examples where BFS returned a longer route than A* (distance mode)

From	To	BFS route	Dist.	A* route	Dist.
Math_Section	Literature	Math → Entrance → Literature	20	Math → Reference → Literature	14
Math_Section	History	Math → Entrance → Literature → History	27	Math → Reference → Literature → History	21

From	To	BFS route	Dist.	A* route	Dist.
Physics_Section	Literature	Physics → Math → Entrance → Literature	26	Physics → Math → Reference → Literature	20
Literature	Math_Section	Literature → Entrance → Math	20	Literature → Reference → Math	14
Literature	Physics_Section	Literature → Entrance → Math → Physics	26	Literature → Reference → Math → Physics	20

These results illustrate the theory: BFS finds fewest-edge routes and can be longer in distance, while A* with an admissible heuristic never was (72 of 72). They also show that in this small graph A* explored fewer nodes on average (3.49) than BFS or DFS (4.50 each). The differences are small because the graph is tiny, and no conclusion about larger graphs should be drawn from them. In time mode, A* explored more nodes than in distance mode for most targets (Table 15, fifth column), as expected when the heuristic is replaced by zero.

4.4 Multi-Book Collection Scenarios

Table 18 lists scenarios run directly against `plan_collection_route`, and M1, M2 and M3 were also run through the browser; the browser outputs matched the direct calls (Figure 10, Figure 11, Figure 12). All scenarios start from the Entrance in distance mode unless stated.

Table 18: Multi-book scenarios

No.	Collection (priority, shelf)	Limit / authorised	Route	Cost / explored	Collected; missed
M1	OS Concepts (Low, CS); AI (High, AI); Calculus (High, Math); World War II (High, History)	100 / no	Entrance → CS → AI → Physics → Math → Reference → Literature → History → Literature → Entrance → CS	83 / 18	All four, in the order AI, Calculus, WWII, OS; none missed
M2	Same as M1	30 / no	Entrance → CS → AI → CS	20 / 5	AI, OS; missed Calculus, WWII
M3	Intro. to Algorithms (High, CS); Confidential Neural Networks (High, Restricted); Calculus (High, Math)	∞ / no	Entrance → CS → Entrance → Math	28 / 4	Algorithms, Calculus; missed Confidential Neural Networks
M4	Same as M3	∞ / yes	Entrance → CS → Restricted_Archive → Math	45 / 5	All three
M5	Intro. to Algorithms; AI; Quantum Mechanics (all High)	∞ / no, time mode	Entrance → CS → AI → Physics	27 / 6	All three
M6	Three books all shelved at CS_Section (High, High, Low)	∞ / no	Entrance → CS	10 / 1	All three (second and third cost 0)
M7	Sapiens (Low, History); AI (High, AI); Encyclopedia (High, Reference)	∞ / no	Entrance → CS → AI → CS → Entrance → Reference → Literature → History	50 / 12	All three; Low book last
M8	Intro. to Algorithms; To Kill a Mockingbird; Quantum Mechanics; Encyclopedia (all High)	60 / no	Entrance → CS → Entrance → Literature → Reference → Math → Physics	52 / 10	First three; missed Encyclopedia
M9	Five High books (Reference, CS, AI, Physics, History) — the five-book default-limit case	100 / no	Entrance → Reference → Entrance → CS → AI → Physics → Math → Reference → Literature → History	64 / 15	All five
M10	Empty collection	∞ / no	Entrance	0 / 0	None (button in the UI shows an alert instead)

Scenario M1 shows the greedy behaviour clearly: although the Low-priority book (OS Concepts) was added first and is shelved next to the Entrance, it is collected **last**, after a long walk through Physics, Math, Reference, Literature and History, because the planner serves High-priority books first. The planner also revisits sections (for example Literature

appears twice, and the walk ends by returning to `CS_Section`). Scenario M6 shows that books sharing a shelf add no cost once the shelf is reached. Scenario M7 shows an example in which the priority rule produces a detour: the nearby Reference book is collected after the AI book and before the Low-priority book, so the route walks to AI and back before returning to Reference. These are consequences of a priority-ordered greedy strategy; the planner makes no attempt to find the shortest overall route, so the routes are valid but not necessarily optimal.

4.5 Priority and Time/Cost Limit Analysis

To study the limit, the collection of scenario M1 (AI, Calculus, WWII — all High — and OS Concepts — Low) was planned with different limits (Table 19). The unconstrained cost of the full plan is 83.

Table 19: Effect of the time/cost limit on the M1 collection (direct calls)

Limit	Cost used	Collected	Missed
0	0	—	AI, Calculus, WWII, OS
9 to 14	8	Calculus	AI, WWII, OS
15 to 19	15	AI	Calculus, WWII, OS
20 to 32	20	AI, OS	Calculus, WWII
33 to 53	33	AI, Calculus	WWII, OS
54 to 82	54	AI, Calculus, WWII	OS
83 and above	83	AI, Calculus, WWII, OS	—

Three behaviours follow directly from the code. **First,** a high-priority book that does not fit is skipped, and the planner continues: with a limit of 9 the High-priority AI book (cost 15) is skipped, yet the High-priority Calculus book (cost 8) is collected; with a limit of 20 the Low-priority OS book is collected after AI because it is close. **Second,** the outcome is not monotonic in the limit in the sense of “collected set grows by adding books”: a limit of 32 collects OS (and not Calculus), whereas a limit of 33 collects Calculus (and not OS), since Calculus is processed earlier and consumes the budget that OS would have used. **Third,** the limit applies to the cumulative outward route and excludes the walk back to the Entrance, so the figure reported (for example 83) is the cost of reaching the last book, not of a round trip. In the interface, the red message “Out of Time! Missed:” lists every uncollected book, so it also appears for books missed for other reasons.

4.6 Restricted Access and Closed-Section Scenarios

Table 20: Restricted-access and closed-section scenarios

Scenario	How tested	Observed result
Restricted goal, unauthorised, A*	Direct call; API	No path, cost ∞, 8 nodes explored. The API returns `Infinity` and the browser cannot parse it (see Figure 14).
Restricted goal, authorised, A*	Direct call	Path Entrance → CS_Section → Restricted_Archive, cost 25, 7 explored.
Restricted goal, unauthorised, BFS and DFS	Direct call	Paths returned (BFS cost 25, DFS cost 31); the restriction is ignored.
Restricted book in a multi-book plan, unauthorised	Direct call; browser	Book reported as missed; the plan continues with the other books (M3, Figure 12).
Restricted book in a multi-book plan, authorised	Direct call; API with `authorized: true`	Collected (M4). The browser never sends this flag, so this is unreachable from the interface.
Only a restricted book, unauthorised	API	HTTP 200, route [Entrance], cost 0, book listed as uncollected.

Scenario	How tested	Observed result
Closed section CS_Section, target AI_Section, A*	Direct call	Rerouted via Math_Section and Physics_Section: cost 26, 5 explored.
Closed section Math_Section, target Physics_Section, A*	Direct call	Rerouted via CS_Section and AI_Section: cost 27, 5 explored.
Closed section equal to the goal	Direct call	No path, cost ∞.
Closed sections from the user interface or API	Inspection and runs	Not possible: the API passes an empty list and no control exists.

The closed-section rule therefore works in the algorithms (the search avoids the closed node, and a closed goal is unreachable) but cannot be activated by a user. The “crowded section” behaviour listed in the README has no counterpart in the code and could not be tested.

4.7 Unit Test Results

The submitted tests were run with `python -m unittest test_algorithm.py -v` in the unmodified `python-files` folder. Five tests ran: **one passed and four errored**, so the submitted suite does not currently pass.

Table 21: Results of the submitted unit tests

Test	Result	Cause
`test_bfs_finds_path`	Pass	BFS returned a path from Entrance to AI.
`test_astar_finds_optimal_path`	Error	`ValueError: too many values to unpack (expected 3)` at line 31.
`test_closed_sections_avoided`	Error	Same error at line 37.
`test_restricted_access_denied`	Error	Same error at line 43.
`test_restricted_access_granted`	Error	Same error at line 48.

All four errors have one cause: `astar` returns four values (path, cost, explored count, closed list), while the tests unpack three. As a **supplementary check outside the submitted suite**, the same four scenarios were run on the tests’ own five-node graph with the return value unpacked correctly; the intended assertions held (Table 22). This shows that the A* logic matches what the tests intend; it does not change the fact that the delivered tests fail as submitted. The suite also has no DFS test, no multi-book test, no API test and no test of an unreachable BFS target.

Table 22: Supplementary check of the four erroring tests (four return values unpacked; not part of the submitted suite)

Intended assertion	Observed	Holds?
Cost from Entrance to AI is 15	Path Entrance → CS → AI, cost 15	Yes
CS closed: route uses Math and avoids CS	Path Entrance → Math → AI, cost 15	Yes
Unauthorised access to Restricted: no path	Path None, cost ∞	Yes
Authorised access to Restricted: path ends there	Path Entrance → CS → AI → Restricted, cost 20	Yes

4.8 Observations

- The graph model and A* work as intended on the supplied data: every reachable target was reached with the least distance, with the Euclidean heuristic admissible for all 12 edges.
- BFS and DFS work as search functions but cannot be used from the interface and ignore restrictions. DFS produced routes more than four times as long as A* (63 against 14 for `Physics_Section`).
- The planner behaves as the report describes: priority-ordered, greedy, bounded by a cumulative limit, and tolerant of unreachable books.
- Alerts recorded in the browser run: “Book is unavailable.” (adding an already-collected book) and, in the source, “Search for a book first!” and “Add a book to collection first.”. The last two are as written in `script.js`; they were not all triggered in the run.
- The `explored` figure returned by the planner covers only accepted legs, and the closed list of A* is unordered.
- The pairwise comparison supports, on this small graph, the textbook relationship between the algorithms; it should not be generalised.

4.9 Limitations Observed

Table 23 lists the defects and limitations found. They are reported as observed and have not been fixed in the code or hidden in the descriptions above.

Table 23: Defects and limitations observed in the submitted project

No.	Finding	Evidence and effect
1	BFS and DFS fail through the API	`app.py` lines 41 and 44 unpack three values from two-value functions; HTTP 500 in every test.
2	A* failure produces invalid JSON	Unreachable target: `"cost":Infinity`; browser parse error; no message shown.
3	Unit tests fail as submitted	4 of 5 tests error because `astar` returns four values.
4	BFS and DFS ignore restricted nodes and `authorized`	Restricted archive reachable by BFS/DFS; DFS routes pass through it.
5	Closed sections not exposed	Parameter exists, but no API field, UI control or data uses it.
6	Crowded sections not implemented	No code; the UI label only selects time weights, which equal distances.
7	README says books are graph nodes	Only the browser draws book nodes; the back-end graph has 9 nodes.
8	README says open and closed lists are displayed	Only an unordered closed list is returned.
9	Authorisation unreachable from the UI	The browser never sends `authorized`; restricted books are always missed.
10	No input validation	Unknown target → HTTP 500 (`KeyError`); empty limit → `null` → HTTP 500 (`TypeError`).
11	Single-book mode uses only the last-added book	`script.js` line 183.
12	Search shows only the first match	E.g. “Artificial Intelligence” shows one book even though a second title contains the phrase.
13	“Out of Time!” label is imprecise	Also displayed for books missed because of restrictions.
14	Greedy planner, no optimality	Priority-ordered; rejected books are never retried; return trip not counted.
15	Medium priority and unavailable books not exercised	No such records in the data.
16	External dependency and unpinned CDN script	Vis.js from unpkg without version; page needs internet access.
17	Debug mode in the start script	`app.run(debug=True)`; suitable for development only.

5 Conclusion

This project addressed the problem of finding books in a library and planning a trip that collects several of them. The approach was to model the library as a weighted graph, to implement three classical search algorithms (BFS, DFS and A*) with a Euclidean heuristic, and to build a goal-based planner that applies A* repeatedly in priority order within a cost limit. A Flask back end, a JSON knowledge base and a Vis.js interface make the system visible and interactive.

Execution of the submitted code supports the following outcomes. A* returned least-distance routes for all reachable targets and explored fewer nodes on average than BFS or DFS in a pairwise comparison on the nine-node graph. BFS returned fewest-edge routes that were longer in distance for 12 of 72 pairs, and DFS returned routes that were often much longer, which matches the theory. The multi-book planner collected books in priority order, skipped books that exceeded the limit, rejected restricted books without authorisation and kept going afterwards.

The execution also showed that the delivered project is incomplete in several respects: BFS and DFS fail through the web interface, unreachable A* targets break the response format, four of five unit tests error, closed sections cannot be set by a user, crowded sections are not implemented, and authorisation cannot be supplied from the interface. These findings are as valuable to a reader as the successes, because they show the gap between a documented design and a tested implementation.

As an educational system the project demonstrates state-space search on a concrete example, the difference between uninformed and informed search, the role of an admissible heuristic, the effect of constraints such as restricted and closed locations, and the limits of a greedy, priority-based planner compared with an optimal one. It is a clear small-scale illustration of how AI search ideas carry over to an everyday navigation task.

Future Enhancements

> **Note.** Everything in this section is a **proposed improvement**. None of it has been implemented in the submitted project or in this report's experiments.

- **Extend the map.** Move the graph and coordinates to a configurable file that can describe floors, aisles and individual shelves, and validate on load that every edge distance is at least the straight-line distance, so the heuristic stays admissible.
- **Dynamic obstacles and crowding.** Let sections be closed or congested at run time and add a cost penalty for crowded sections, exposed through the API and interface; implement the closed-section parameter end to end.
- **Better multi-book optimisation.** Compare the greedy planner with nearest-neighbour ordering, dynamic programming for small collections, or a heuristic improvement step, and allow the user to choose between priority-first and shortest-trip strategies.
- **Accessibility and interface design.** Add keyboard navigation, screen-reader labels, colour-blind-safe route colours, a step-by-step textual route and a mobile layout, and show the correct reason when a book is missed.
- **Persistent collections.** Store collection lists and user permissions in a database with authentication, so that the restricted-shelf rule can be applied to real users.
- **Stronger automated tests.** Correct the return-value mismatch, add tests for DFS, the planner, the API routes, error responses and every edge of the data, and run them automatically on every change.
- **Better reporting of cost and search behaviour.** Return ordered open and closed lists, per-leg costs and the return-to-entrance cost, return valid JSON error objects for unreachable targets, and let the interface replay the search step by step.

Bibliography

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[2] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[4] Pallets, "Flask documentation." [Online]. Available: https://flask.palletsprojects.com/

[5] vis.js community, "vis-network documentation." [Online]. Available: https://visjs.github.io/vis-network/docs/network/

[6] Python Software Foundation, "heapq — Heap queue algorithm," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/heapq.html

[7] Python Software Foundation, "math — Mathematical functions," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/math.html

[8] T. Bray, Ed., "The JavaScript Object Notation (JSON) data interchange format," RFC 8259, Internet Engineering Task Force, Dec. 2017. [Online]. Available: https://www.rfc-editor.org/info/rfc8259

[9] MDN Web Docs, "Fetch API." [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[10] Python Software Foundation, "unittest — Unit testing framework," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/unittest.html

[11] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[12] G. Gutin and A. P. Punnen, Eds., *The Traveling Salesman Problem and Its Variations*. New York, NY, USA: Springer, 2007.

[13] Microsoft, "Playwright for Python documentation." [Online]. Available: https://playwright.dev/python/